University Resource Allocation System

BOOKBBK



MSc Computer Science Project Report

Author:
Jun Bin Yap

Supervised by:
Oded Lachish

Submitted on:
September 2024

Abstract

This report details the development and features of a resource allocation system, **BOOKBBK**[1] that provides venue booking functionalities for clients, and resource allocation for university staff. To maximise profit generated from venue hiring, this project focuses on using various algorithms to allocate halls to clients' booking requests. Resource manager can either accept the allocation results or re-run the process, and existing clients are allowed to check their booking status at any time. For every hall, its allocation results are displayed in a comprehensive calendar view.

Acknowledgement

I hereby declare that the content presented in this document is entirely my own work with reference to existing literature and research. While external tools, such as Grammarly, were used to proofread and enhance the clarity of the text, all ideas, concepts, and expressions are original and have not been plagiarised from any other source.

[1] **BOOKBBK** demo is now live on `https://bookbbk.netlify.app/`!

Contents

Page

1. Introduction **5**
- 1.1. Background . . . 5
- 1.2. Related Software . . . 6
  - 1.2.1 Timetable Scheduling Assistant . . . 6

2. System Overview **8**
- 2.1. Client Interaction . . . 8
  - 2.1.1 Make A Booking . . . 8
  - 2.1.2 Check Booking Status . . . 9
  - 2.1.3 Re-schedule Failed Requests . . . 10
- 2.2. Resource Manager Interaction . . . 11
  - 2.2.1 Viewing and Allocating Bookings . . . 11
  - 2.2.2 Lecture Hall Table and Calendar . . . 12

3. System Design and Architecture **13**
- 3.1. System Architecture . . . 13
- 3.2. Software Architecture . . . 15
  - 3.2.1 Presentation Layer . . . 15
  - 3.2.2 Application Logic Layer . . . 15
  - 3.2.3 Data Access Layer . . . 15
- 3.3. Database Schema . . . 15

4. Resource Allocation Algorithms **17**
- 4.1. Greedy Algorithms . . . 17
  - 4.1.1 Earliest Start Time . . . 18
  - 4.1.2 Longest Booking Duration . . . 19
- 4.2. Random Allocation . . . 20
- 4.3. Backtracking Algorithm . . . 21
- 4.4. Integer Linear Programming . . . 23

5. Evaluation of Algorithms **25**
- 5.1. Case 1: Longer Duration Greedy Does Not Guarantee Higher Profits . . . 25
  - 5.1.1 Outcome - Greedy Longest Duration . . . 26

5.1.2 Outcome - Backtracking Algorithm . . . 27
5.1.3 Summary . . . 27
5.2. Case 2: Earliest Start Time Does Not Ensure Maximum Profit . . . 28
5.2.1 Outcome - Greedy Earliest Start Time . . . 29
5.2.2 Outcome - Greedy Longest Duration . . . 30
5.2.3 Outcome - Backtracking Algorithm . . . 31
5.2.4 Summary . . . 31
5.3. Case 3: Increasing Number of Bookings and Time Complexity . . . 32
5.3.1 Test 1 - 15 Bookings and 5 Halls . . . 32
5.3.2 Test 2 - 20 Bookings and 5 Halls . . . 32
5.3.3 Test 3 - 30 Bookings and 5 Halls . . . 33
5.3.4 Test 4 - 25 Bookings and 10 Halls . . . 33
5.3.5 Summary . . . 33
5.4. Case 5: Comparing profits generated by ILP and Backtracking . . . 34
5.4.1 Test 1 . . . 34
5.4.2 Test 2 . . . 34
5.4.3 Test 3 . . . 35
5.5. Summary . . . 35

6. Tools and Technology **36**
6.1. Technology Stack . . . 36
6.2. Frontend Development . . . 36
6.3. Backend Development . . . 36
6.4. Database Technology . . . 37
6.5. Version Control and Deployment . . . 37

7. Testing **38**
7.1. Test-Driven Development Approach . . . 38
7.1.1 Unit Tests . . . 38
7.1.2 Functional Testing . . . 38
7.2. Automated Testing Pipeline . . . 39

8. Conclusion **40**
8.1. Personal Reflection . . . 40
8.2. Future Steps . . . 40

References **41**

9. Appendix A **42**
9.1. Precedent Software . . . 42
9.1.1 Timetable - Queen Mary University of London . . . 42
9.1.2 MS Teams Room Finder - Birkbeck, University of London . . . 43
9.2. Software Development Methodology and Process . . . 44

9.2.1 Sprint 0 - Development Environment 44
9.2.2 Sprint 1 - GUI Development 44
9.2.3 Sprint 2 - Server-side Development 44
9.2.4 Sprint 3 - Relational Database 45
9.2.5 Sprint 4 - Implementing Allocation Algorithms 45
9.2.6 Sprint 5 - Deploying BOOKBBK Online 46
9.2.7 Database Deployment 46
9.2.8 Front-end Application Deployment 46
9.2.9 Back-end Application Deployment 46
9.3. Running Algorithms Using GUI 47
9.3.1 Backtracking Algorithm 47
9.3.2 Greedy - Earliest Start Time 47
9.3.3 Greedy - Longest Duration 48
9.3.4 Random Allocation 49
9.4. Sample Bookings For 5.4 50
9.5. ILP Setup 51
9.5.1 Hall Time Constraints 51
9.5.2 Example Booking as Variable 52
9.5.3 Example Solver Model 52
9.6. CI/CD Pipeline 53

10. Appendix B **54**

Project Related List of Keywords

- **ILP** - Integer Linear Programming. Used to create one of the allocation algorithms for this project.
- **Client** - A BOOKBBK customer, unless when used in the context of system/software architecture components.
- **GUI** - The Graphical User Interface built using React.js.
- **AWS** - Amazon Web Service. EC2 is used for this project.
- **VM** - Virtual Machine deployed for this project, powered by AWS EC2.
- **CI/CD** - Continuous Integration and Deployment. Refers to the automated testing-and-deployment pipeline set up using **GitHub Actions**.

1. INTRODUCTION

1.1 Background

SPACES like lecture halls not only serve as academic venues but also offer opportunities for revenue generation. Universities with surplus space often hire out these venues, creating a valuable income stream[1].

Traditionally, manual allocation of spaces is not only time-consuming but error-prone and inefficient. Without automation, it might take weeks or months to construct a timetable. (Burke and Varley 2000), highlighting the need for a modern computer reservation system equipped with an efficient resource allocation algorithm.

In educational institutions, space allocation involves the process of assigning available areas to various activities, each with specific space needs. Typically, an activity is described by its duration and by resources required to process the activity (Müller and Barták 2001). The goal is to create a timetable that meets certain mandatory constraints, while fulfilling additional preferences.

The hall allocation problem here is similar to well-known scheduling problems, which have been proven to be NP-complete (Garey 1979). While it is easy to verify a proposed solution, finding the best one is computationally difficult. For instance, there are many possible ways to schedule bookings (activities), but only a few of the results will satisfy all the constraints such as capacity and no time-overlaps.

A related problem is the Nurse Scheduling Problem (NSP), which has been researched since 1976 (Miller, Pierskalla, and Rath 1976). NSP involves assigning shifts to nurses while balancing hard constraints (e.g., hospital regulations) and soft constraints (e.g., personal preferences), using automated and statistical methods. Similarly, space allocation in universities must balance strict requirements with optional preferences.

Several studies have explored advanced algorithms for university timetabling, including genetic algorithms, integer programming, and tabu search. These approaches offer varying trade-offs in terms of complexity and performance (Alghamdi et al. 2020). While these research focused on algorithmic approaches to timetabling, this project adopted a more practical approach by not only exploring allocation algorithms but also developing a software solution for space management.

[1] University College London's Venue Hiring Page: `https://www.ucl.ac.uk/estates/our-services/conference-and-venue-hire`

1.2 Related Software

UniTime is designed to create and manage course and exam timetables to minimise timetable conflicts. To create a timetable, they utilise CPSolver, a constraint programming-like framework to model the timetabling problem given a set of constraints and requirements[2].

1.2.1 Timetable Scheduling Assistant

Users can enter their course requests ordered by priority and run an automated scheduling.

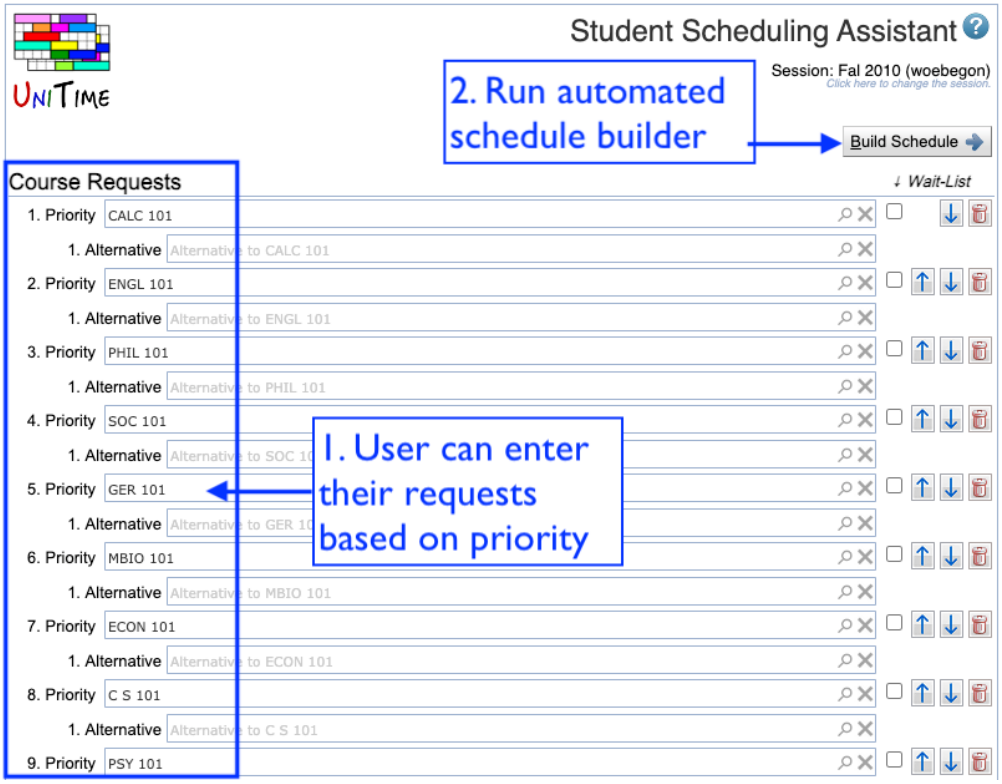
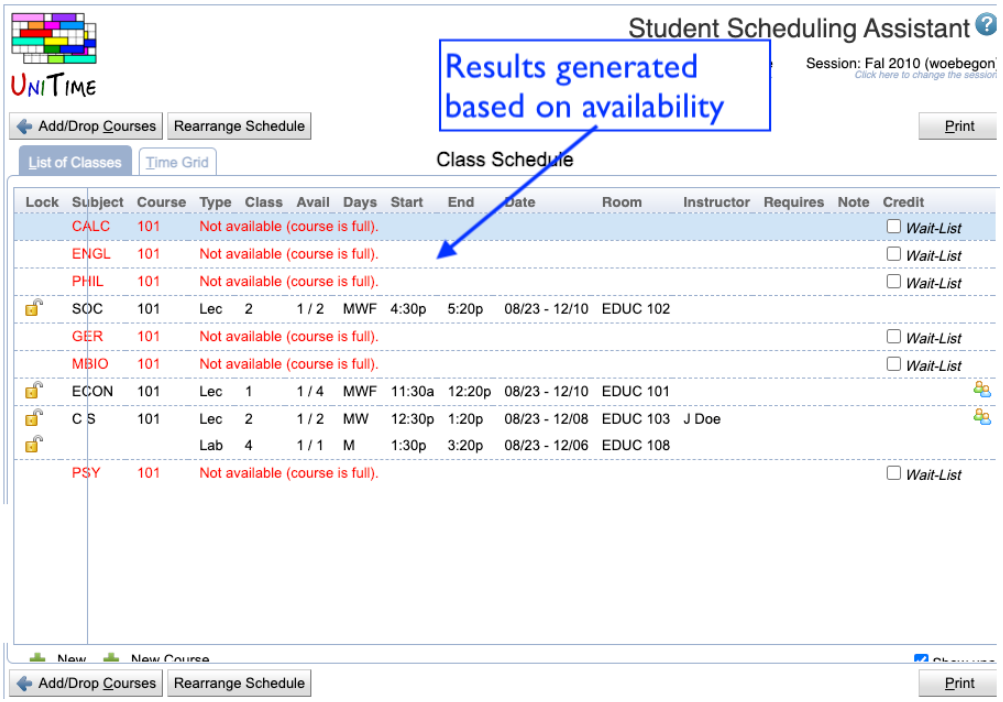
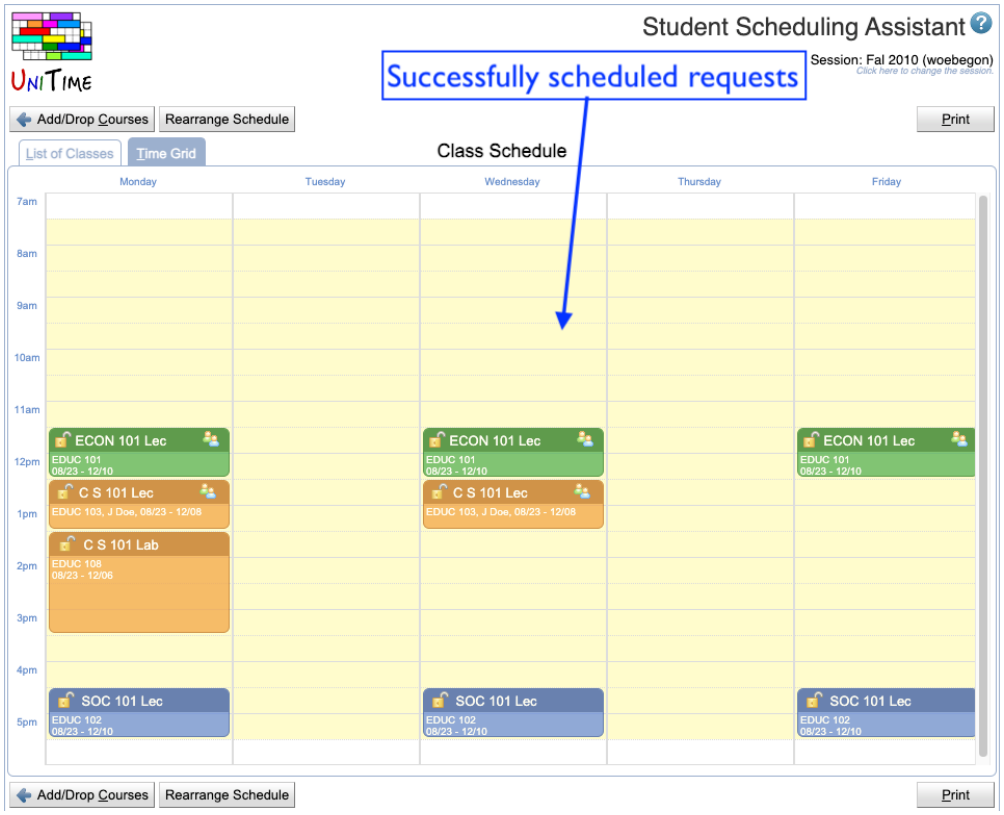


Figure 1.1 *A form for user to enter their desired classes to attend.*

[2]See `https://www.cpsolver.org/present/patat22-unitime.pdf`



(a) *Successful and failed result shown in list view.*



(b) *Successful result shown in calendar view*

Figure 1.2 *Scheduled results can be viewed in list or calendar mode.*

See section 9.1 for other existing software used as references for this project.

2. SYSTEM OVERVIEW

Client reservations are made prior to the start of each university's academic term, ensuring that a finalised schedule for the next 3 months is available before classes begin. The system manages a number of lecture halls, each with varying capacities.

2.1 Client Interaction

2.1.1 Make A Booking

A client can create a booking request specifying their requirements, such as capacity, date, and time. Once the request form is submitted, it will be validated by the front- and back-end applications, and then recorded in the database. A client ID will be generated for new clients upon successful booking submission.

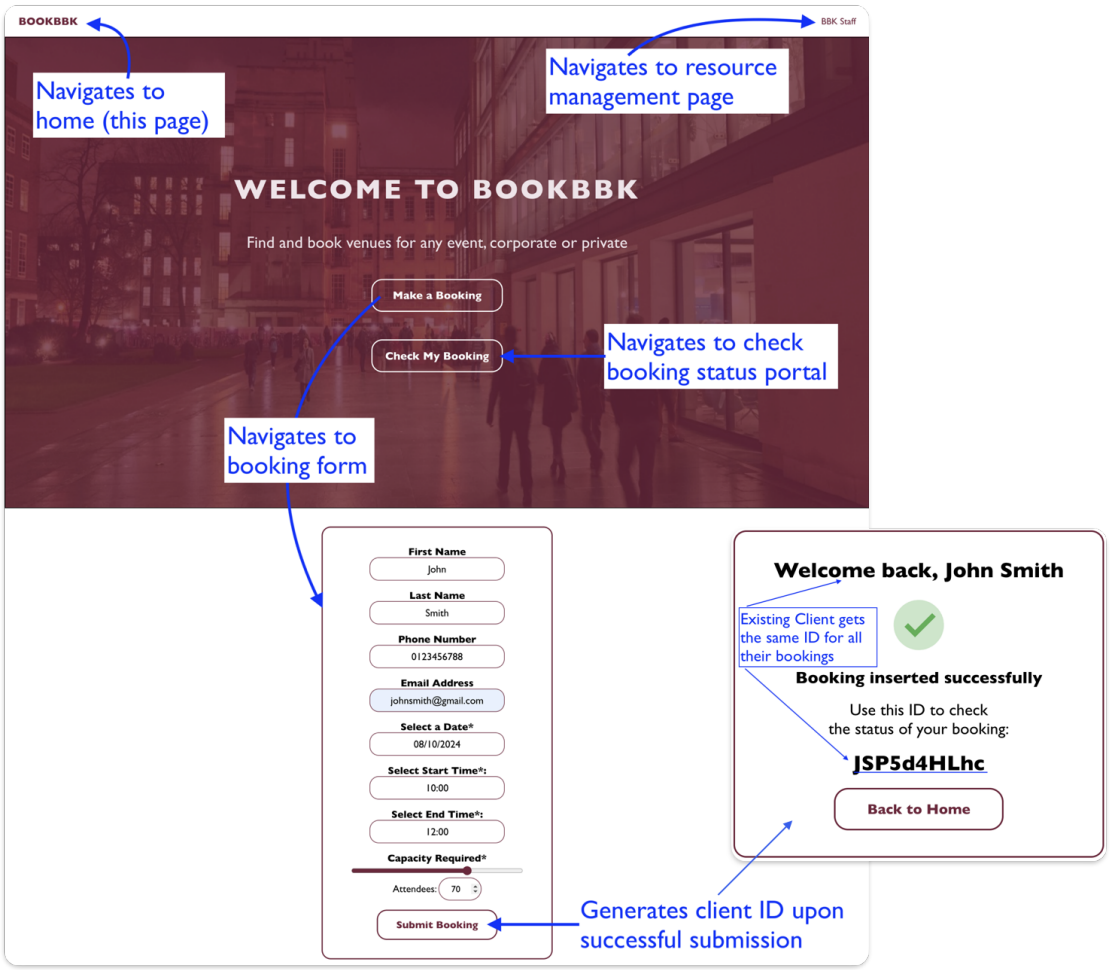


Figure 2.1 ***Main Landing Page*** *A booking portal with various navigations for different users.*

2.1.2 Check Booking Status

Clients can check their booking status using their **email address** and **client ID** obtained upon a successful booking submission. They can view all their reservations, and choose to accept the assigned bookings if they are satisfied with it.

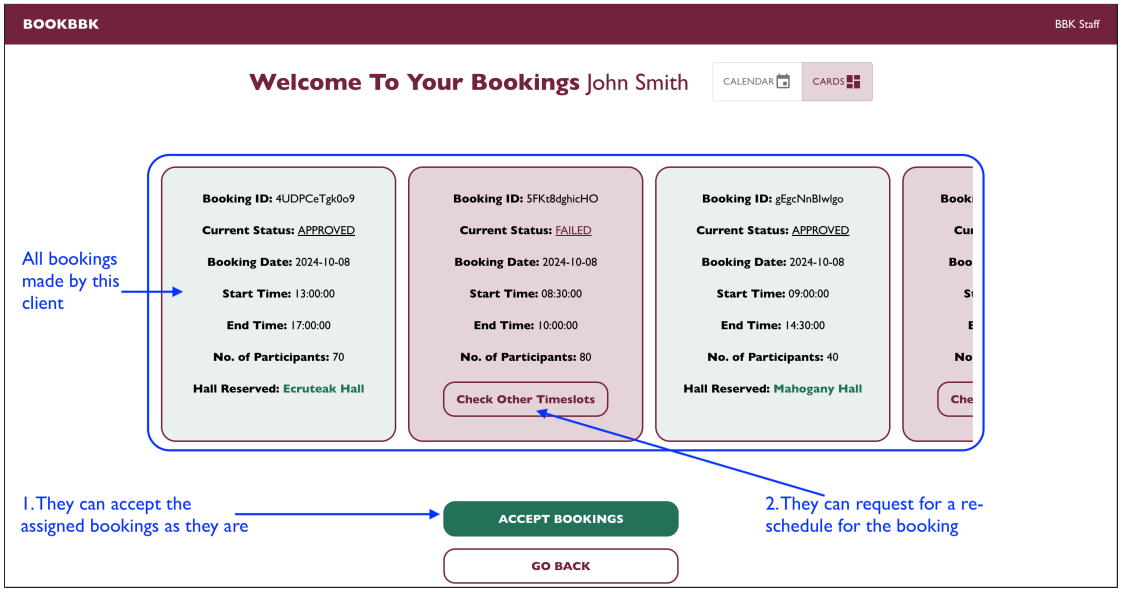
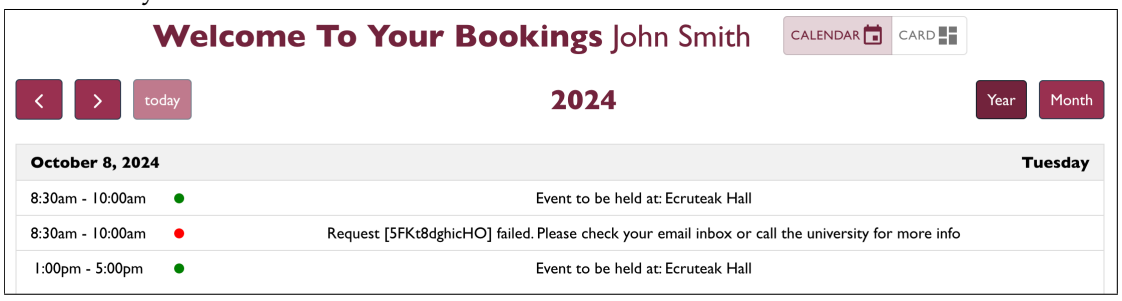


Figure 2.2 *Client can view all their current bookings and submit further inquiries*



(a) Monthly event mode

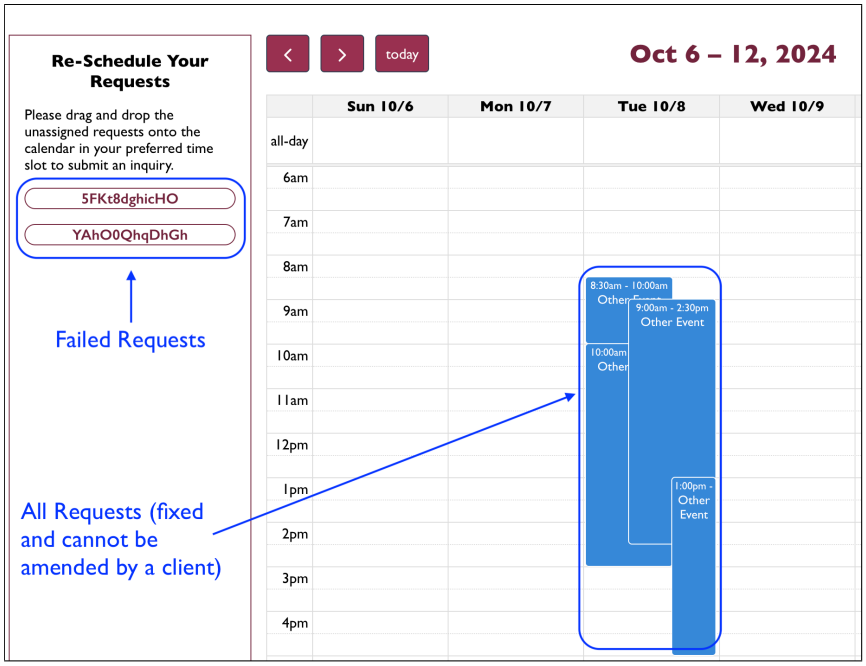


(b) Overall event list mode

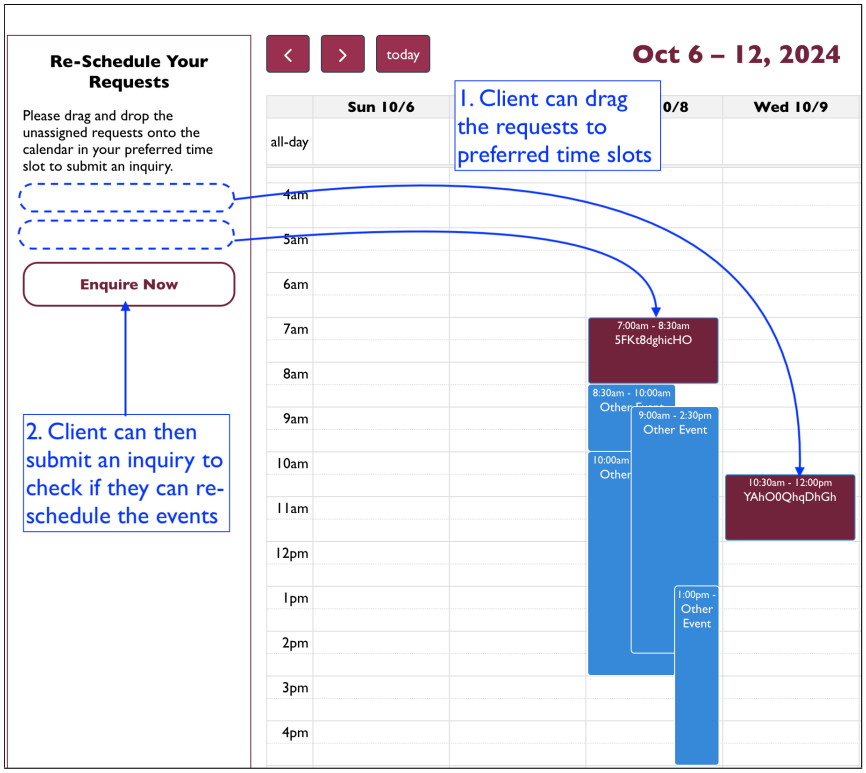
Figure 2.3 *Different calendar view mode*

2.1.3 Re-schedule Failed Requests

Clients can submit an inquiry to re-schedule the failed bookings using an interactive calendar. They can only place the bookings that on time slots that are not occupied by all other already allocated events.



(a) *Calendar Timeline View*



(b) *Interacting with the calendar*

Figure 2.4 ***Reschedule Booking GUI:*** Client can request a rescheduling for their unassigned bookings

2.2 Resource Manager Interaction

A resource manager can oversee all data retrieved from the database in separate tables. Each table provides different information and functionality.

2.2.1 Viewing and Allocating Bookings

A resource manager has the authority to run an automated allocation process using various algorithms. After the automated allocation is complete, they can review the results and select the best allocation based on their judgment.

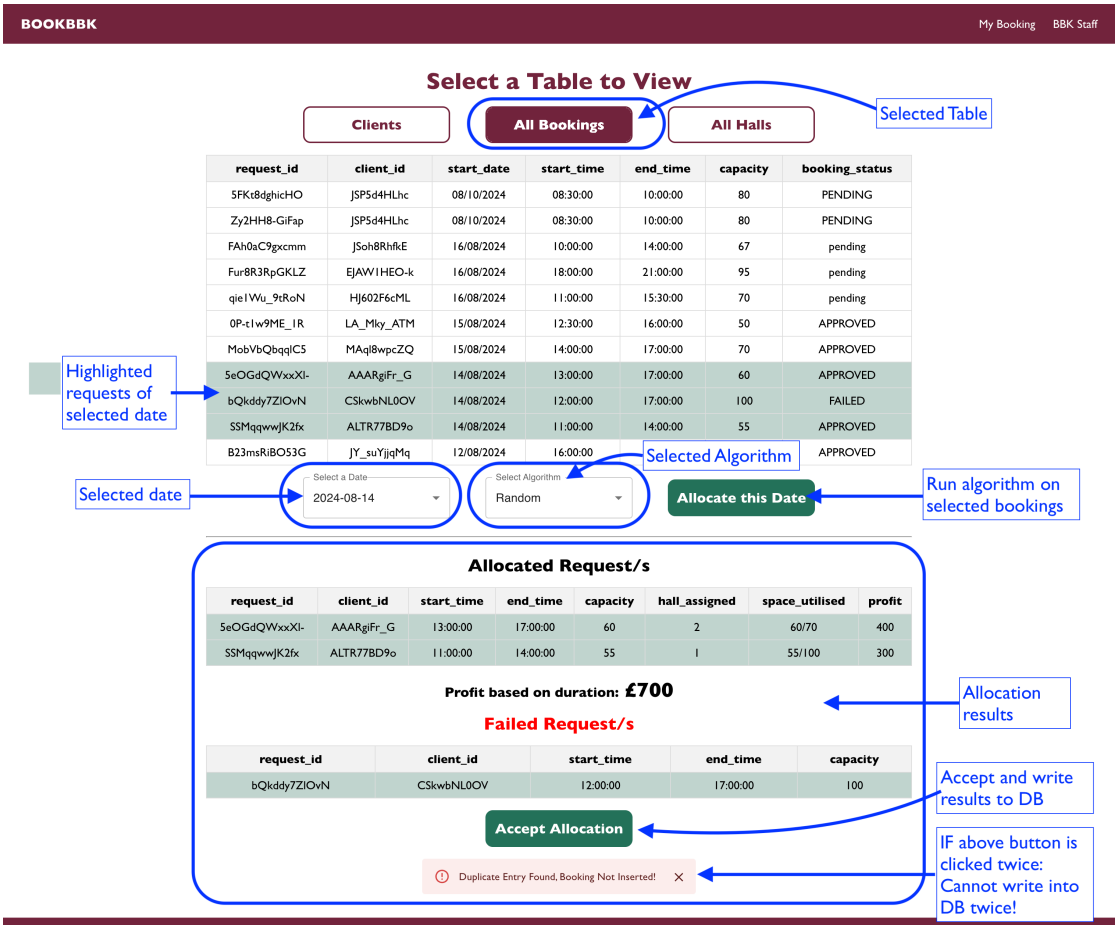


Figure 2.5 ***Resource Management GUI:*** Interface for allocating lecture halls to booking requests.

2.2.2 Lecture Hall Table and Calendar

A resource manager can view allocated requests displayed as calendar events for each hall.

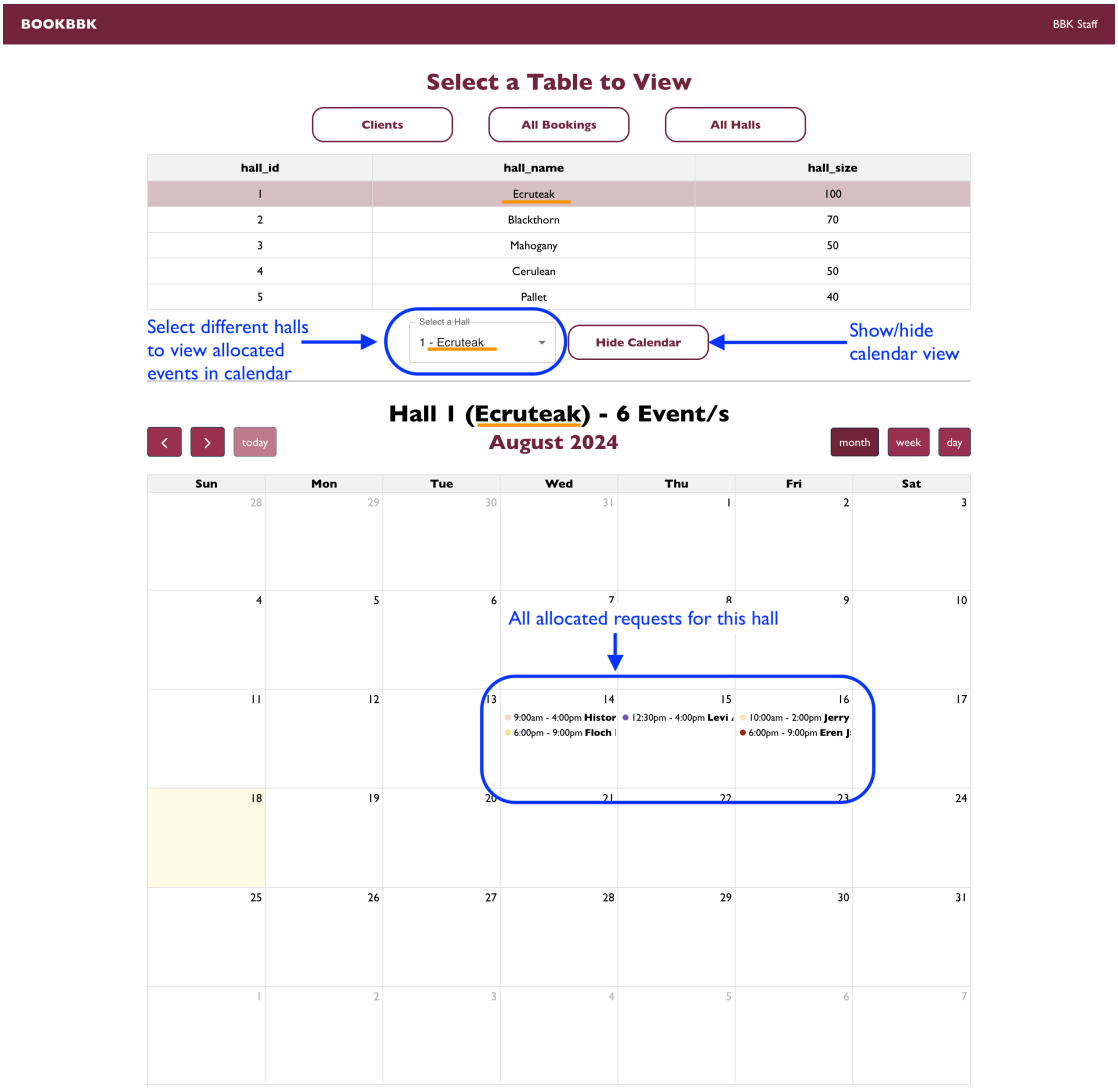


Figure 2.6 ***Lecture Hall Calendar GUI:*** Interface for viewing and allocated events based on halls.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1 System Architecture

BOOKBBK is a web-based application designed to enable booking and management of lecture halls. Different online servers such as **Netlify**, **Render** and **AWS EC2** are used in this project to host the application.

1. **Graphical User Interface (GUI)**: Accessible through web browsers, the GUI allows different users to interact with the system differently.
    - **Client** - They can access the booking portal and make a new booking submission. They can also check their existing booking status by entering their email address and client ID.
    - **Resource Manager** - They can access the resource management page as staff, and run resource allocation.
2. **Application Servers**:
    - **Front-End Server** - Hosts the front-end application that returns static web pages and dynamic components to the web browser. Makes API calls to the back-end server.
    - **Back-End Server** - Hosts the back-end application that runs allocation algorithms and database communication.
3. **Database Server**: An EC2 instance is deployed as a server to run a relational database that stores and persists application data such as lecture halls, bookings, and client information.
4. **Data Communication**: API calls using HTTP/HTTPS requests enabled browser-server communication, while SQL queries are used for server-database interactions. The results from database queries and API responses are represented as JSON objects.

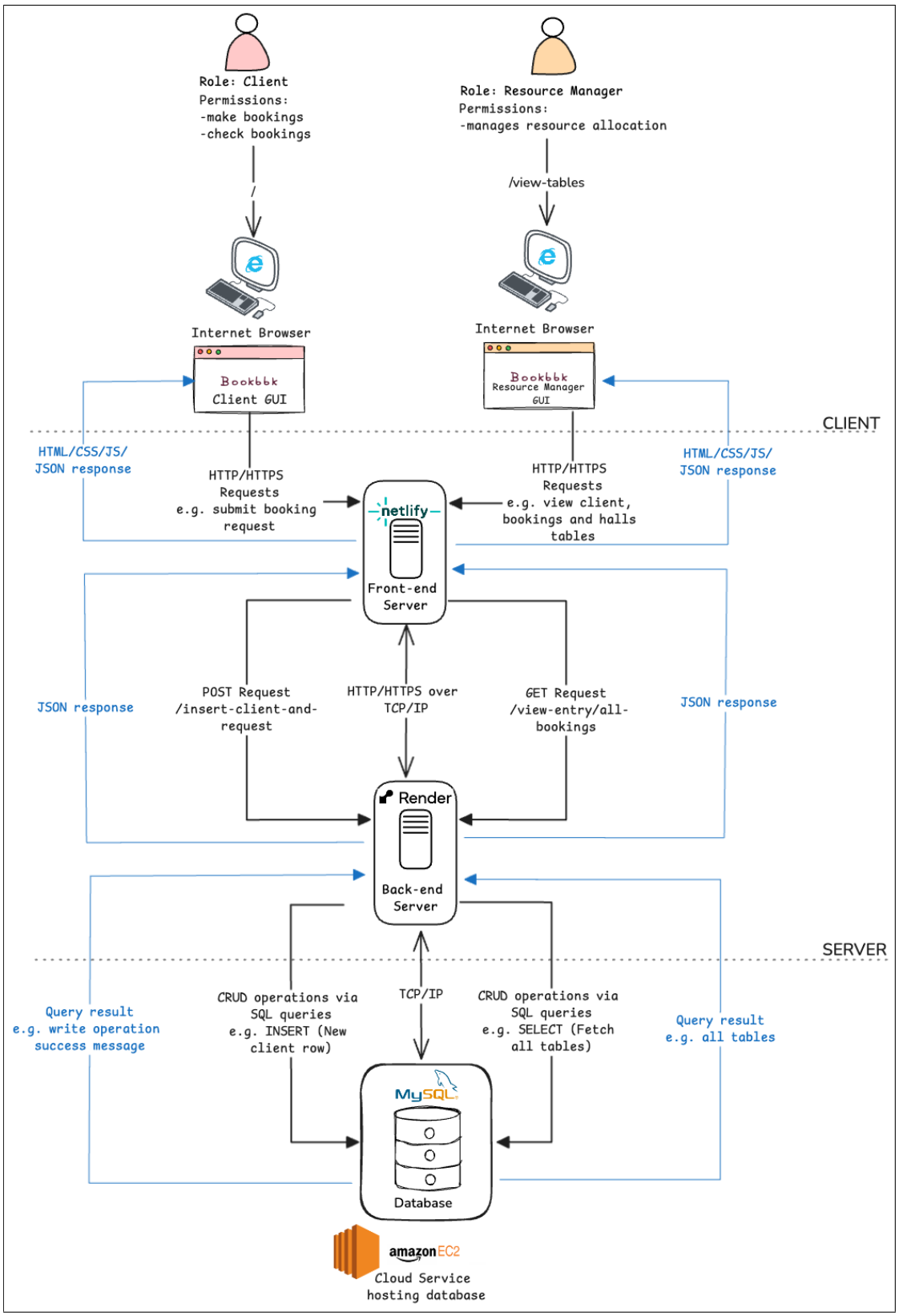


Figure 3.1 *A* high-level view of BOOKBBK's system architecture

3.2 Software Architecture

BOOKBBK employs an **N-tier architecture** which consists of a presentation layer, application logic layer and data access layer. This architecture style was chosen for its benefits including:

- **Improved Maintainability:** The codebase is organised into distinct layers, making it easier to manage, update, and debug without cross-layer interference. For example, additional libraries can be installed for a single tier without having to redesign the whole solution.
- **Controlled Data Flow:** Data moves sequentially through each layer. For instance, the presentation layer interacts with the logic and data access layer rather than making direct database calls, adhering to the principle of separation of concerns.
- **Better Security:** Sensitive information is stored and processed in the logic layer which is isolated from the presentation tier. Exposing information in the presentation tier may introduce security risks such as SQL injection attacks.

3.2.1 Presentation Layer

This is the entry point to the system. Users with different roles can interact with differentiated user interfaces using a web browser. Clients can submit a booking request on the home page, whereas only a university manager can access a resource management page.

3.2.2 Application Logic Layer

With external libraries and user-defined functions, this layer processes background tasks like user authentication, database connection and running allocation algorithms.

3.2.3 Data Access Layer

This layer defines all the functions required to execute database queries. When the logic layer requests for data, a function call will be made in a simple data agnostic way (Kumar 2010).

3.3 Database Schema

The core components of this schema include the following tables:

- **client**: This table maintains the personal details of clients.
- **booking_request**: This table tracks booking requests made by all clients. Each request is uniquely identified by a `request_id` and linked to the client table through the `client_id` foreign key.

- **lecture_hall**: This table contains information about the available lecture halls. It stores the size of each hall, enabling the allocation algorithm to match booking requests with appropriate halls.
- **allocated_bookings**: Once the allocation process is done and the results are accepted by the resource manager, the allocated requests get written into this table. It contains the information of each booking with their assigned halls. This table is essential for managing confirmed bookings and ensuring that hall allocations are properly documented.
- **bbk_staff**: This table does not link to any other tables and stores data for staff members who manage the booking system. It is used for implementing role-based access control, ensuring that only authorised personnel can view and manage hall bookings.

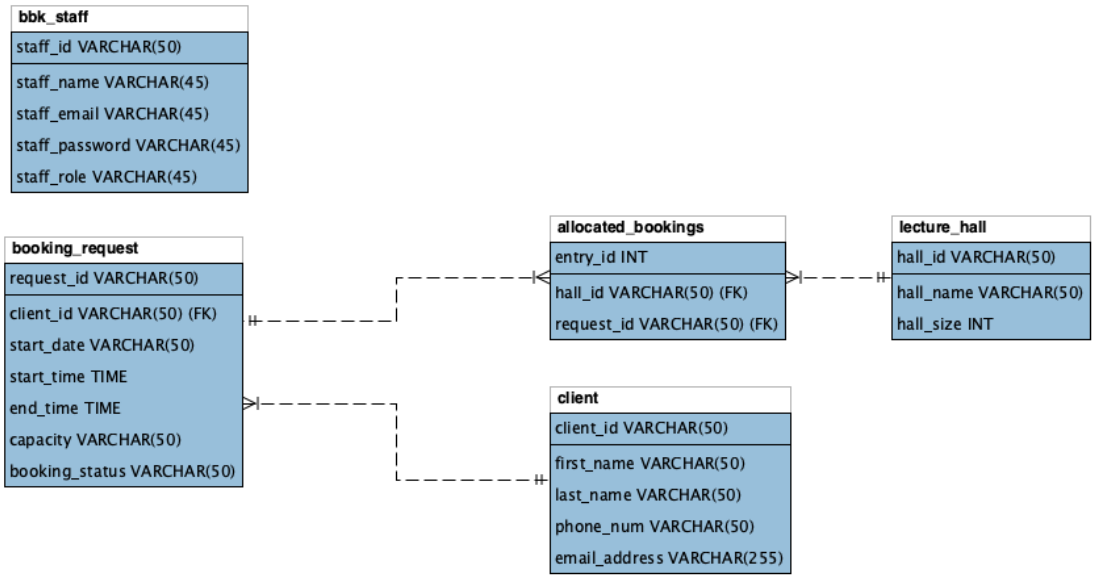


Figure 3.2 *BOOKBBK Database Schema - First row of each table describes its primary key.*

4. RESOURCE ALLOCATION ALGORITHMS

The development of allocation algorithms started with using **simplified examples** visualised in this section. The main assumptions for running the allocation algorithms are:

1. The longer the booking duration, the higher its profit.
2. There are different halls available for the bookings. (denoted as `hallArray` in pseudocode below)
3. There are sample bookings to be assigned. (denoted as `bookingArray` in pseudocode below)
4. **Hall Availability Check:** A helper function `isHallAvailable` determines whether the hall is free during the requested time slot.
    - **Capacity:** The hall's capacity must be equal to or greater than the request's capacity.
    - **Availability:** A hall cannot be double-booked (i.e. no time-clashing allowed).

The core of greedy and random algorithm remains the same, only varies in how the `bookingArray` is sorted.

4.1 Greedy Algorithms

Greedy algorithms in this project works on specific criteria, such as the earliest start time or the longest booking duration. Similar to a *job-sequencing problem* (GeeksforGeeks 2023), these algorithms prioritise bookings by sorting them based on these criterias. The halls were always sorted in ascending order based on their capacities so that smaller halls can be allocated to bookings with fewer attendees first. However, this approach does not guarantee maximum profit, as it may allocate halls to seemingly profitable bookings that ultimately block out other requests with higher revenue potential.

4.1.1 Earliest Start Time

This algorithm sorts and allocates booking requests based on the earliest start time.

Algorithm 1 Earliest Start-time Greedy

```
1:  Input: bookingArray, hallArray
2:  Output: best allocation of requests to halls, total profit, failed requests
3:  function TIMEGREEDY(bookingArray, hallArray)
4:      sortedBookings ← SORTBOOKINGSBYSTARTTIME(bookingArray)
5:      sortedHalls ← SORTHALLSBYCAPACITY(hallArray)
6:      allocatedRequests ← []
7:      failedRequests ← []
8:      for each request in sortedBookings do
9:          isAllocated ← false
10:         for each hall in sortedHalls do
11:             if hall.capacity ≥ request.capacity and ISHALLAVAILABLE( )then
12:                 allocatedRequest ← ASSIGNREQUESTTOHALL(request, hall)
13:                 allocatedRequests.append(allocatedRequest)
14:                 isAllocated ← true
15:                 break
16:             end if
17:         end for
18:         if not isAllocated then
19:             failedRequests.append(request)
20:         end if
21:     end for
22:     totalProfit ← CALCULATETOTALPROFIT(allocatedRequests)
23:     return (allocatedRequests, failedRequests, totalProfit)
24: end function
```

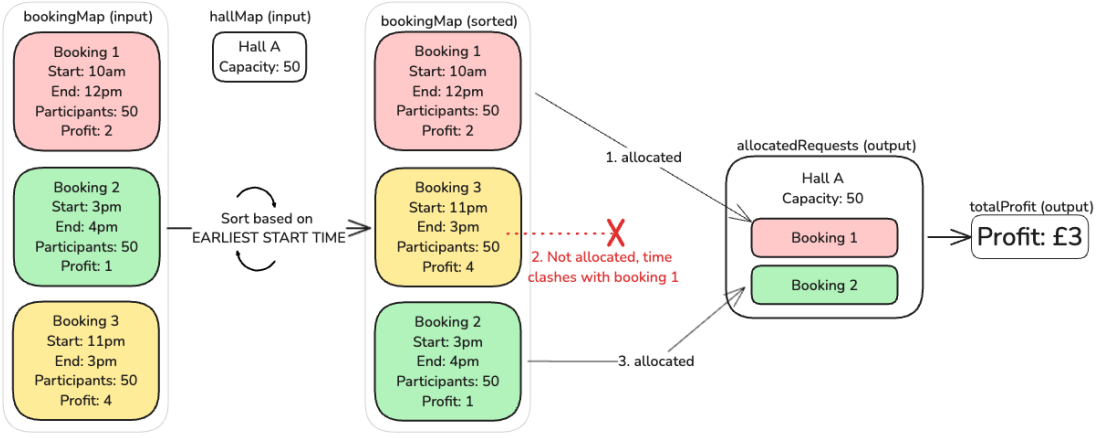


Figure 4.1 *How greedy algorithm works based on earliest start time*

Figure above shows that Booking 3 was not allocated despite having a profit higher than the combined of Booking 1 and 2.

4.1.2 Longest Booking Duration

This algorithm hopes to achieve maximum profit by prioritising bookings with extended time periods.

Algorithm 2 Longest Duration Greedy

```
1:  Input: bookingArray, hallArray
2:  Output: best allocation of requests to halls, total profit, failed requests
3:  function STARTTIMEGREEDY(bookingArray, hallArray)
4:      sortedBookings ← SORTBOOKINGSBYDURATION(bookingArray)
5:      sortedHalls ← SORTHALLSBYCAPACITY(hallArray)
6:      allocatedRequests ← []
7:      failedRequests ← []
8:      for each request in sortedBookings do
9:          isAllocated ← false
10:         for each hall in sortedHalls do
11:             if hall.capacity ≥ request.capacity and ISHALLAVAILABLE( )then
12:                 allocatedRequest ← ASSIGNREQUESTTOHALL(request, hall)
13:                 allocatedRequests.append(allocatedRequest)
14:                 isAllocated ← true
15:                 break
16:             end if
17:         end for
18:         if not isAllocated then
19:             failedRequests.append(request)
20:         end if
21:     end for
22:     totalProfit ← CALCULATETOTALPROFIT(allocatedRequests)
23:     return (allocatedRequests, failedRequests, totalProfit)
24: end function
```

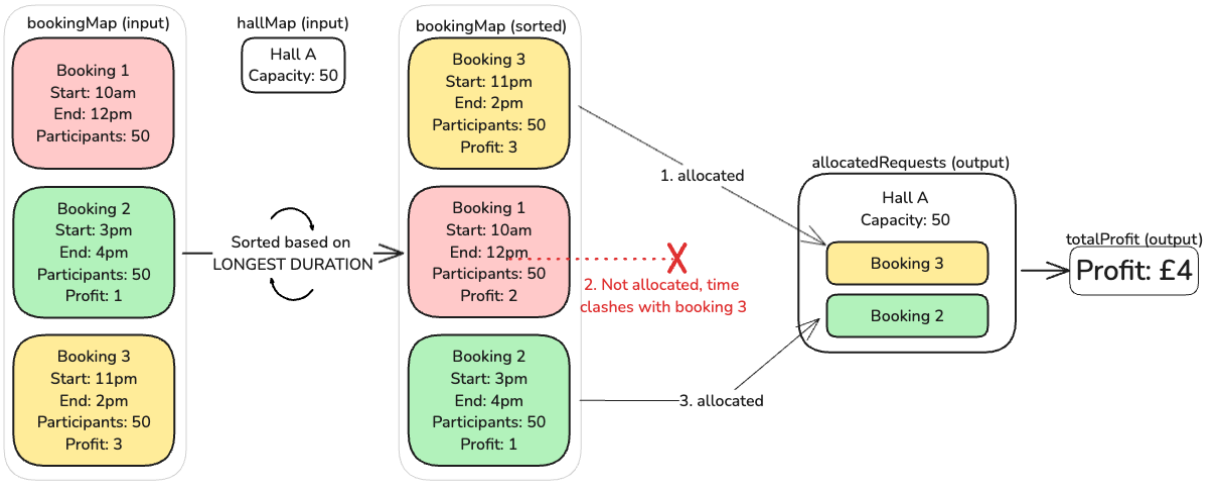


Figure 4.2 *How greedy algorithm works based on longest booking duration*

4.2 Random Allocation

Compared to the previous two algorithms, the only difference is that **both bookings and halls are randomly shuffled** before each run (line 4 and 5). Therefore, profits generated using the sample booking scenario as above will differ each run.

Algorithm 3 Random Assignment Algorithm

```
1:  Input: bookingArray, hallArray
2:  Output: allocation of requests to halls, total profit, failed requests
3:  function RANDOMASSIGNMENT(bookingArray, hallArray)
4:      shuffledBookings ← SHUFFLEARRAY(bookingArray)
5:      shuffledHalls ← SHUFFLEARRAY(hallArray)
6:      allocatedRequests ← []
7:      failedRequests ← []
8:      for each request in shuffledBookings do
9:          isAllocated ← false
10:         for each hall in shuffledHalls do
11:             if hall.capacity ≥ request.capacity and ISHALLAVAILABLE( )then
12:                 allocatedRequest ← ASSIGNREQUESTTOHALL(request, hall)
13:                 allocatedRequests.append(allocatedRequest)
14:                 isAllocated ← true
15:                 break
16:             end if
17:         end for
18:         if not isAllocated then
19:             failedRequests.append(request)
20:         end if
21:     end for
22:     totalProfit ← CALCULATETOTALPROFIT(allocatedRequests)
23:     return (allocatedRequests, failedRequests, totalProfit)
24: end function
```

While unreliable in real world application, **there is a chance to discover more favourable allocations that other more deterministic algorithms might miss.**

4.3 Backtracking Algorithm

This algorithm uses a recursive function to traverse all potential allocations. It covers the allocation results that the previous algorithms are not able to achieve.

Algorithm 4 Backtracking Algorithm

```
1: Input: index, allocatedRequests, bookingArray, hallArray
2: Output: best allocation of requests to halls, total profit, failed requests
3: function ALLOCATERECURSIVE(index, allocatedRequests, bookingArray,
   hallArray)
4:
5:     if index ≥ length(bookings) then
6:         totalProfit ← CALCULATETOTALPROFIT(allocatedRequests)
7:         return {allocatedRequests, totalProfit, failedRequests}
8:     end if
9:
10:    availableHalls ← ∅
11:    request ← bookings[index]
12:
13:    for each hall in hallArray do
14:        if hall.capacity ≥ request.capacity and ISHALLAVAILABLE( )then
15:            availableHalls.push(hall)
16:        end if
17:    end for
18:
19:    skippedResult ← ALLOCATERECURSIVE(index + 1, allocatedRequests,
   bookingArray, hallArray)
20:    bestAllocation ← skippedResult.allocatedRequests
21:    bestProfit ← skippedResult.totalProfit
22:    bestUnallocated ← skippedResult.failedRequests
23:
24:    for each hall in availableHalls do
25:        allocatedRes ← {request details with hall info and profit}
26:        allocatedRequests.push(allocatedRes)
27:        result ← ALLOCATERECURSIVE(index+1, allocatedRequests,
   bookingArray, hallArray)
28:        if result.totalProfit > bestProfit then
29:            bestProfit ← result.totalProfit
30:            bestAllocation ← result.allocatedRequests
31:            bestUnallocated ← result.failedRequests
32:        end if
33:        allocatedRequests.pop
34:    end for
35:    return {bestAllocation, bestProfit, bestUnallocated}
36: end function
```

1. The function operates with four primary inputs: `index`, `allocatedRequests`, `bookingArray` and `hallArray`.

   - **Index:** This represents the current booking being considered for allocation.
   - **AllocatedRequests:** This array stores the current state of allocations that have been made during the recursion. Initally an empty array.

   The base case is triggered when the current index exceeds the number of bookings, indicating that all bookings have been processed. At this point, the total profit is calculated, and the current allocation is returned as the result.

2. **Hall Availability Check:** For each booking request, the helper function `isHallAvailable` determines whether the hall is free during the requested time slot. If a hall is available, it is added to the `availableHalls` list for further consideration.

3. **Branching** The function explores two primary branches for each booking:

   - **Skipping the Current Request:** This branch considers what happens if the current booking is not allocated to any hall. The algorithm makes a recursive call with the next booking in the list (index + 1), leaving allocatedRequests unchanged.
   - **Allocating the Current Request:** For each hall in the availableHalls list, the algorithm attempts to allocate the current booking. A new allocation record is created, including details such as the hall assigned, space utilised, and profit. The function then recursively proceeds with this new allocation added to allocatedRequests.

4. **Profit Maximisation:** As the recursive calls return results, the algorithm compares the profits of different branches. It selects the allocation that yields the highest total profit. This process continues until the recursion unwinds to the initial call, at which point the best possible allocation is identified.

4.4 Integer Linear Programming

To overcome the long computing time of backtracking algorithm (see section 5.3) while getting an optimal allocation, a linear programming solution is created using the `jsLPSolver` package.

The hall allocation task with its requirements can be formulated as an ILP problem:

- **An objective** - To maximise the revenue and space utilisation for a hall.
- **Decisions need to be made under constraints** - Hall capacity and time overlap as constraints.
- **All decision variables must be integers** - Each booking is either allocated or not allocated.

Algorithm 5 Integer Linear Programming

```
1:  Input: bookingArray, hallArray
2:  Output: best allocation of requests to halls, total profit, failed requests
3:  function LPSOLVE(bookingArray, hallArray)
4:      Sort halls by capacity in ascending order
5:      Initialise empty list of allocated requests
6:
7:      for each hall in sortedHalls do
8:          Create a new optimization model to maximise profit and capacity
9:
10:         Filter unallocated bookings
11:         Filter bookings to match hall capacity
12:
13:         if there are matching bookings then
14:             Define capacity constraints for the hall
15:             Parse the bookings into ILP-compatible format
16:             Set constraints, variables, and integer conditions in the model
17:
18:             Solve the ILP model
19:             Select best combination of bookings based on profit and capacity
20:
21:             for each selected booking do
22:                 Assign the booking to the hall
23:                 Calculate and store its profit
24:             end for
25:         end if
26:     end for
27:     Return allocated requests, total profit, and unallocated requests
28: end function
```

The function `lpSolve` implements ILP to solve the allocation problem. Here's how it works:

1. **Sorting Halls by Capacity** - The halls are sorted by their capacity to ensure that the smallest halls are considered first to prevent smaller bookings being allocated to a large hall.

2. **Creating the ILP Model** - For each hall, an optimisation model is created[1], where the objective is to **maximise both profit and capacity utilisation**. See section 9.5 for how the constraints are defined and how a solver model is set up.

3. **Filtering and Parsing Bookings** - The bookings that have not been allocated yet are filtered to check whether their required capacity fits the hall. The `parseBookings` function converts these bookings into a format suitable for the ILP model, preparing constraints and variables for the solver.

4. **Defining Constraints** - The capacity and time-overlap constraints ensure that each booking fits within the hall's capacity, and no hall is double-booked.

5. **Solving and Allocating Bookings** - Once the model is built, the ILP solver runs to find the best combination of bookings.

6. **Handling Unallocated Bookings** - After the allocation process, the algorithm returns a list of allocated requests and calculates the total profit. It also identifies which bookings could not be allocated due to space and time constraints.

The key to successfully utilising this package lies in how time constraints were defined for each hall and the resources required for every booking[2]. For each hall, the available time in a day was divided into 30-minute time slots, which were modelled as resources to be allocated to the variables (bookings). The required time slots for each booking were similarly modelled to align with the format of the constraints, ensuring compatibility between the booking requirements and hall availability.

[1]The linear model is created based on examples shown in the jsLPSolver repository: `https://github.com/JWally/jsLPSolver`

[2]See code for implementing ILP: `https://github.com/junyap95/university-resource-allocation/blob/master/src/services/lpSolver.js`

5. EVALUATION OF ALGORITHMS

Algorithms from chapter 4 were executed, compared and evaluated across different use cases [1]. All bookings to be allocated are on the same date.

5.1 Case 1: Longer Duration Greedy Does Not Guarantee Higher Profits

Scenario: Consider eight bookings that can be conceptually divided into 2 groups based on their non-overlapping time slots. The first group consists of 2 longer bookings, while the second group includes several shorter bookings whose combined duration slightly exceeds that of the first group. It is assumed that only one hall is available to accommodate all these bookings.

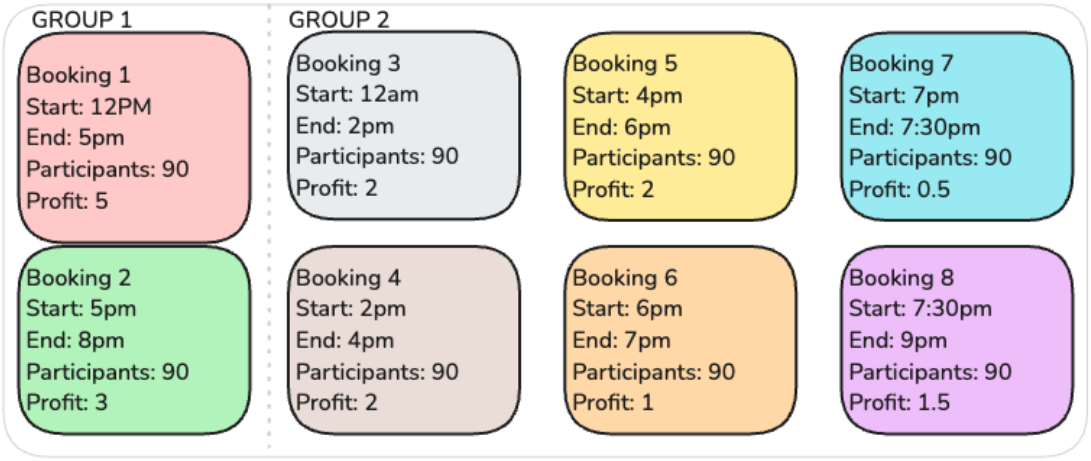
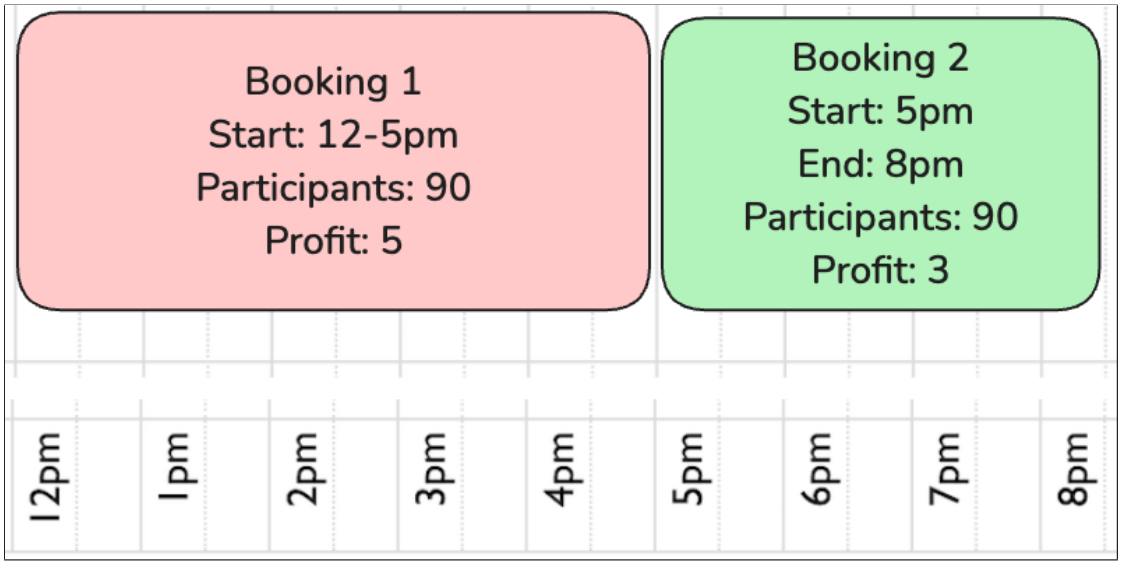


Figure 5.1 *Eight separate bookings with identical capacity and varied in timeslots.*

Longer duration greedy and backtracking algorithm were executed on these bookings. Backtracking algorithm was used here to provide a maximum profit to check if the profit of greedy algorithm fell short.

[1] See also appendix section 9.3 for other generic use cases

5.1.1 Outcome - Greedy Longest Duration



(a) *Booking 1 and 2 are allocated.*

Allocated Request/s

start_time	end_time	capacity	hall_assigned	space_utilised	profit
12:00:00	17:00:00	90	1	90/100	500
17:00:00	20:00:00	90	1	90/100	300

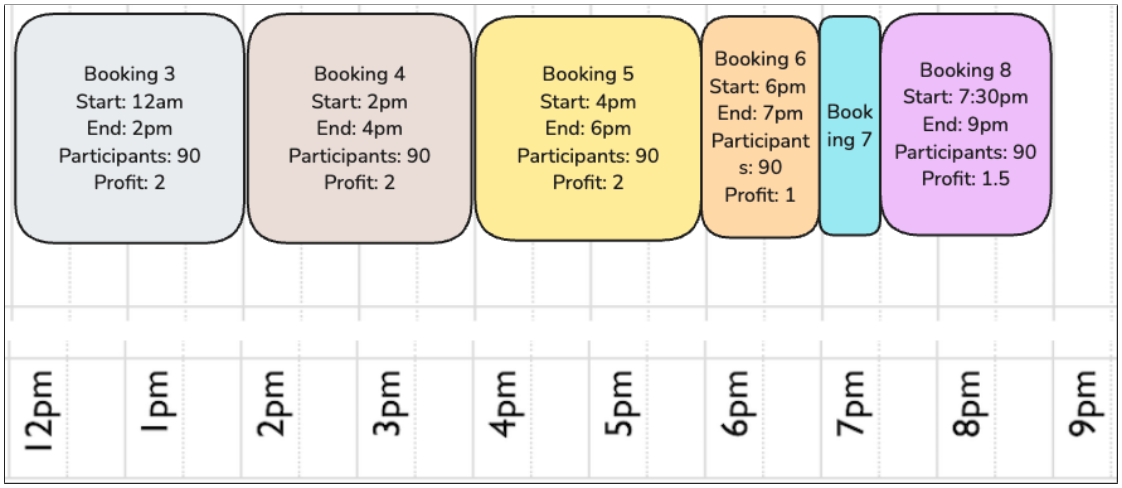
Profit based on duration: £800

Failed Request/s

client_id	start_time	end_time	capacity
RS_ICSP4yRz	12:00:00	14:00:00	90
Y _TEKYKv0z	14:00:00	16:00:00	90
AT_KGpuCS67	16:00:00	18:00:00	90
KH_rolVL1Vl	19:30:00	21:00:00	90
KN_37Bb8ZTp	18:00:00	19:00:00	90
SG_D5wacPgU	19:00:00	19:30:00	90

(b) *Test run in BOOKBBK app - Profit generated: £800*

5.1.2 Outcome - Backtracking Algorithm



(a) *Booking 3 to 8 are allocated.*

Allocated Request/s

start_time	end_time	capacity	hall_assigned	space_utilised	profit
14:00:00	16:00:00	90	1	90/100	200
18:00:00	19:00:00	90	1	90/100	100
16:00:00	18:00:00	90	1	90/100	200
19:30:00	21:00:00	90	1	90/100	150
19:00:00	19:30:00	90	1	90/100	50
12:00:00	14:00:00	90	1	90/100	200

Profit based on duration: £900

Failed Request/s

client_id	start_time	end_time	capacity
SG_caPwpZcK	12:00:00	17:00:00	90
MF_z3rRt0qM	17:00:00	20:00:00	90

(b) *Test run in BOOKBBK app - Profit generated: £900*

5.1.3 Summary

Running the greedy-longer duration algorithm can result in missed opportunities to assign shorter bookings that could potentially yield higher profits.

5.2 Case 2: Earliest Start Time Does Not Ensure Maximum Profit

Similar to case 1, this algorithm might lead to suboptimal utilisation of halls, as earlier bookings might block out later bookings with higher profit.

Scenario: Consider **1 available hall** and **8 bookings**. They are arbitrarily split into 2 groups, where group 1 starts slightly earlier.

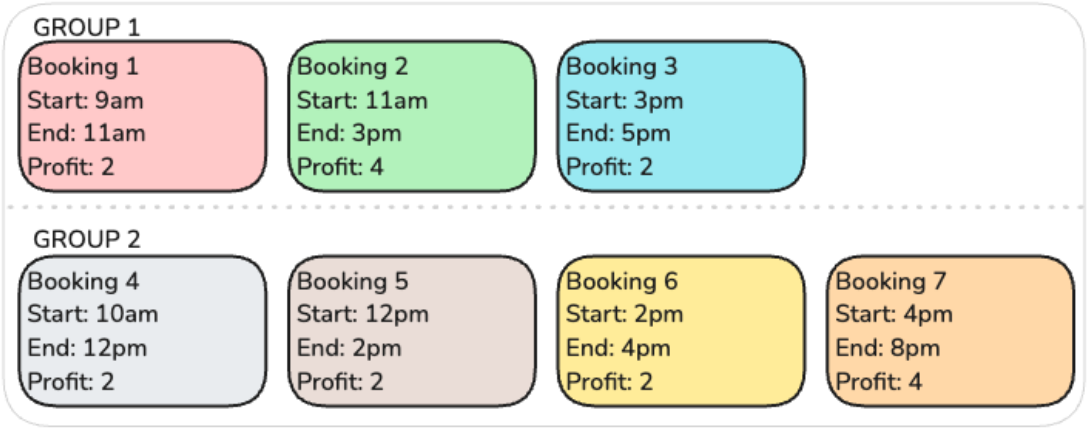
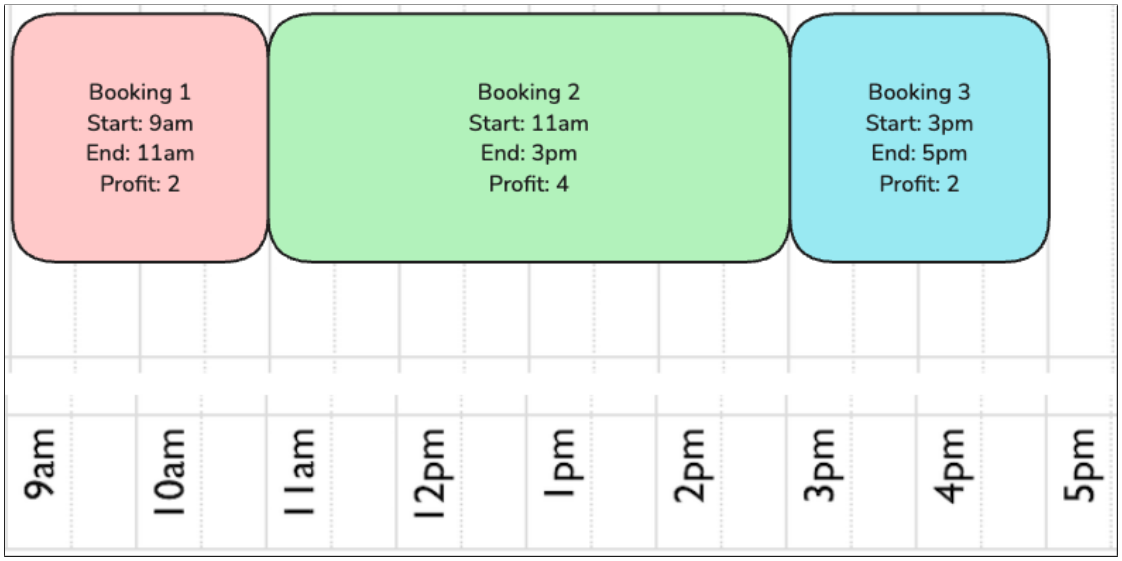


Figure 5.4 *Seven separate bookings to be assigned to one available hall.*

Earliest start-time greedy, longest duration greedy and backtracking algorithm were executed. The results produced by the latter two algorithms were used as comparisons to those of the start-time greedy.

5.2.1 Outcome - Greedy Earliest Start Time



(a) *Booking 1, 2, 3 are allocated.*

Allocated Request/s

start_time	end_time	capacity	hall_assigned	space_utilised	profit
09:00:00	11:00:00	90	1	90/100	200
11:00:00	15:00:00	90	1	90/100	400
15:00:00	17:00:00	90	1	90/100	200

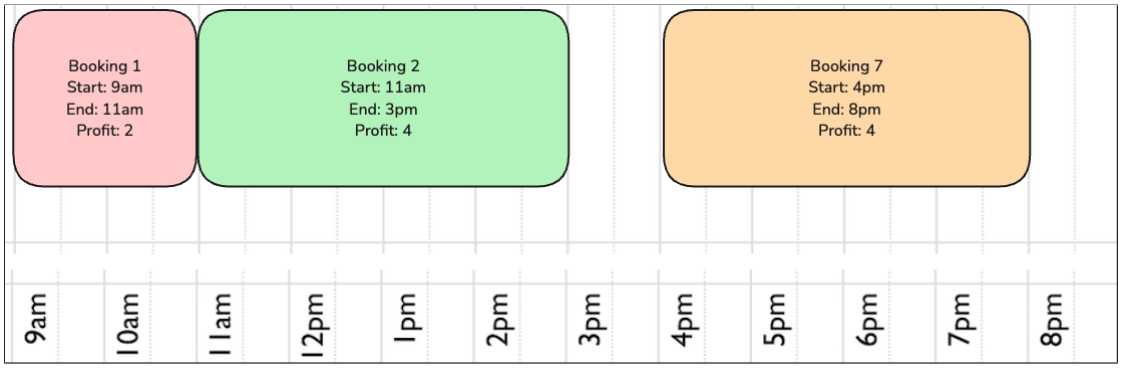
Profit based on duration: £800

Failed Request/s

client_id	start_time	end_time	capacity
CSkwbNL0OV	10:00:00	12:00:00	90
ES_laKJJIG4	12:00:00	14:00:00	90
FG_yBy1hm3K	14:00:00	16:00:00	90
HD_7AlvWUBm	16:00:00	20:00:00	90

(b) *Test run in BOOKBBK app - Profit generated: £800*

5.2.2 Outcome - Greedy Longest Duration



(a) *Booking 1, 2, 7 are allocated.*

Allocated Request/s

start_time	end_time	capacity	hall_assigned	space_utilised	profit
11:00:00	15:00:00	90	1	90/100	400
16:00:00	20:00:00	90	1	90/100	400
09:00:00	11:00:00	90	1	90/100	200

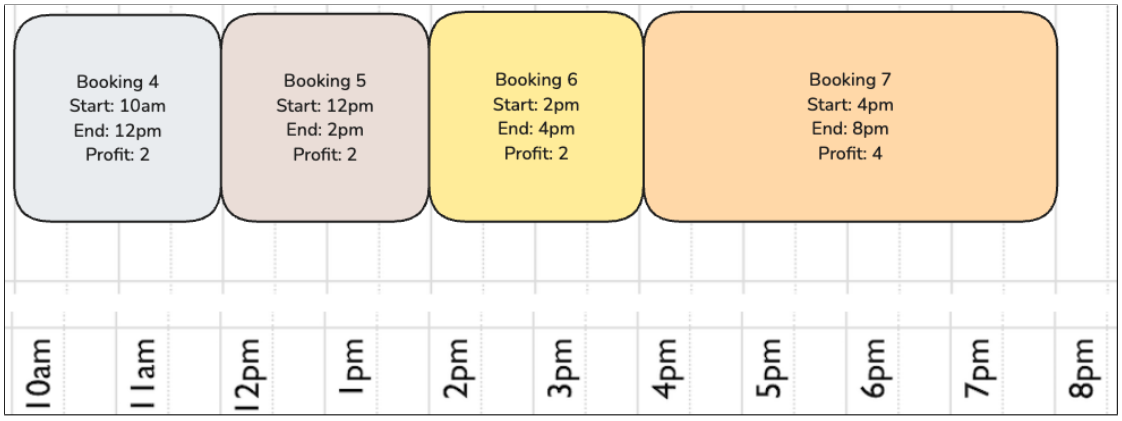
Profit based on duration: £1000

Failed Request/s

client_id	start_time	end_time	capacity
CSkwbNL0OV	10:00:00	12:00:00	90
ES_laKJJIG4	12:00:00	14:00:00	90
FG_yBy1hm3K	14:00:00	16:00:00	90
AT_KGpuCS67	15:00:00	17:00:00	90

(b) *Test run in BOOKBBK app - Profit generated: £1000*

5.2.3 Outcome - Backtracking Algorithm



(a) *Booking 4, 5, 6, 7 are allocated.*

Allocated Request/s

start_time	end_time	capacity	hall_assigned	space_utilised	profit
14:00:00	16:00:00	90	1	90/100	200
16:00:00	20:00:00	90	1	90/100	400
10:00:00	12:00:00	90	1	90/100	200
12:00:00	14:00:00	90	1	90/100	200

Profit based on duration: £1000

Failed Request/s

client_id	start_time	end_time	capacity
SG_D5wacPgU	09:00:00	11:00:00	90
AAARgiFr_G	11:00:00	15:00:00	90
AT_KGpuCS67	15:00:00	17:00:00	90

(b) *Test run in BOOKBBK app - Profit generated: £1000*

5.2.4 Summary

The greedy earliest start time algorithm can overlook later bookings that offer higher potential profits. Both longest-duration greedy and backtracking algorithms performed better in terms of profit generated.

5.3 Case 3: Increasing Number of Bookings and Time Complexity

Each algorithm was tested with increasing number of bookings or halls.

5.3.1 Test 1 - 15 Bookings and 5 Halls

Scenario: Consider **5 available halls** with different capacities and **15 bookings** with overlapping timeslots and varying number of attendees.
Outcome: Both the backtracking and ILP algorithms produced the most profitable allocations. However, ILP proved to be significantly faster.

Algorithm	Time Taken (seconds)	Profit Generated (£)
Greedy - Earliest Start Time	0.001	4000
Greedy - Longest Duration	0.001	4250
Random	0.002	2950
Backtracking	14.37	4350
ILP	0.019	4350

Table 5.1: Runtime and profits generated by various algorithms

5.3.2 Test 2 - 20 Bookings and 5 Halls

Scenario: Consider **5 available halls** with different capacities and **20 bookings** with overlapping timeslots and varying number of attendees.
Outcome: Both ILP and backtracking algorithms obtained highest profit, but the latter took substantially more time (around 8 minutes) to run with only 5 more bookings than test 1.

Algorithm	Time Taken (seconds)	Profit Generated (£)
Greedy - Earliest Start Time	0.001	4000
Greedy - Longest Duration	0.001	4250
Random	0.001	3700
Backtracking	512.33	4400
ILP	0.021	4400

Table 5.2: Runtime and profits generated by various algorithms

5.3.3 Test 3 - 30 Bookings and 5 Halls

Scenario: **5 available halls** with different capacity and **30 bookings** with overlapping timeslots and varying number of attendees.
Outcome: ILP produced a profit slightly lower than the backtracking method which only finished after more than an hour.

Algorithm	Time Taken (seconds)	Profit Generated (£)
Greedy - Earliest Start Time	0.002	4400
Greedy - Longest Duration	0.002	4500
Random	0.001	3650
Backtracking	7757	4800
ILP	0.008	4550

Table 5.3: Runtime and profits generated by various algorithms

5.3.4 Test 4 - 25 Bookings and 10 Halls

Scenario: Consider **10 available halls** with different capacity and **25 bookings** with overlapping timeslots and varying number of attendees.
Outcome: Despite producing the same profits, the ILP method was much faster than the backtracking approach.

Algorithm	Time Taken (seconds)	Profit Generated (£)
Greedy - Earliest Start Time	0.002	4800
Greedy - Longest Duration	0.002	4800
Random	0.001	4550
Backtracking	9043	5000
ILP	0.016	5000

Table 5.4: Runtime and profits generated by various algorithms

5.3.5 Summary

Despite its profitability, the backtracking algorithm can have an unpredictable and lengthy execution time, which varies depending on the machine it runs on. With just 25 bookings, the execution time was quite substantial. Although test 3 suggests that the ILP approach may sometimes generate lower profits compared to backtracking, it is still preferred due to its significantly short runtime. Other algorithms may sometimes produce profitable allocations, but their outcomes are unpredictable and inconsistent.

5.4 Case 5: Comparing profits generated by ILP and Backtracking

Using 3 sets of 15 distinct bookings, allocations produced by both algorithms were compared based on their profits and halls utilisation.

5.4.1 Test 1

Scenario: See Figure 9.8a for sample bookings.
Outcome: ILP produced a profit slightly lower than backtracking.

Metric	Backtracking	ILP
Failed Requests	1	2
Profit Generated (£)	3400	3200
Bookings assigned to Hall 1	4	3
Bookings assigned to Hall 2	3	3
Bookings assigned to Hall 3	3	3
Bookings assigned to Hall 4	2	2
Bookings assigned to Hall 5	1	2

Table 5.5: *Comparison of profits and hall assignment by two algorithms*

5.4.2 Test 2

Scenario: See Figure 9.8b for sample bookings.
Outcome: ILP produced a profit slightly lower than backtracking.

	Backtracking	ILP
Failed Requests	3	4
Profit Generated (£)	3150	2900
Bookings assigned to Hall 1	5	3
Bookings assigned to Hall 2	4	4
Bookings assigned to Hall 3	2	2
Bookings assigned to Hall 4	1	1
Bookings assigned to Hall 5	0	1

Table 5.6: *Comparison of profits and hall assignment by two algorithms*

5.4.3 Test 3

Scenario: See Figure 9.8c for sample bookings.
Outcome: Both algorithms produced the same profit.

	Backtracking	ILP
Failed Requests	2	2
Profit Generated (£)	3700	3700
Bookings assigned to Hall 1	4	3
Bookings assigned to Hall 2	4	3
Bookings assigned to Hall 3	2	4
Bookings assigned to Hall 4	2	2
Bookings assigned to Hall 5	1	1

Table 5.7: *Comparison of profits and hall occupancy by two algorithms*

5.5 Summary

The test results show that ILP can generate allocations comparable to backtracking, though occasionally with slightly lower profit outcomes. Both algorithms allocated bookings across various halls, resulting in similar hall coverage. The choice between ILP and backtracking should be based on the number of bookings. For fewer bookings, backtracking is recommended, while ILP becomes more effective as the number increases (e.g. more than 25 bookings).

6. TOOLS AND TECHNOLOGY

6.1 Technology Stack

- **JavaScript:** Both the front- and back-end applications were written in JavaScript. This reduces development time and complexity by allowing code to be written in a consistent syntax and within a unified ecosystem.
- **SQL:** SQL queries in the MySQL dialect are written as strings in the database access layer to interact with the database, handling tasks such as data retrieval, updates, and transaction management.
- **YAML:** YAML is used for composing configuration files required to run GitHub Actions workflows, which automate the CI/CD process (see section 7.2).

6.2 Frontend Development

- **React.js:** The main landing page, booking form, resource tables, and calendars were created using React.js. Its architecture supported the development of reusable UI components and also provided the use of hooks for state management, which was important in creating responsive and dynamic user interfaces.
- **FullCalendar:** A reusable calendar component is used by different components in the front-end application, such as an event calendar for each lecture halls and a client calendar for re-scheduling.
- **React-datepicker:** This package is used in the booking form to provide an intuitive interface for users to pick dates and times for their booking requests.

6.3 Backend Development

- **Express.js:** Runs computationally intensive tasks in the background to separate application logic from the front-end. It simplifies the creation of RESTful APIs and manages HTTP requests. It provides a flexible framework for using external libraries and building the server-side logic, such as allocation algorithms and database connections.
- **MySQL2:** External library that communicates with the database and allows SQL queries to be written as strings and executed in a JavaScript environment.
- **Playwright:** This library allows automated browser interactions by simulating user actions like filling out a form, clicking the submit button and navigating between pages. It was used for writing end-to-end tests that work across multiple browsers.

- **jsLPSolver:** Allows for the creation of an ILP model to solve hall allocation problem using JavaScript.

6.4 Database Technology

MySQL was selected as it requires little complexity to set up locally and online.

- **Data Integrity:** Booking requests were correctly associated with the appropriate lecture halls by using foreign key constraints.
- **Complex Queries and Relationships:** Complex queries can be made to write or retrieve data for displaying dynamic front-end components. To obtain allocated events and other relevant information would require the execution of the following code:

```
1       const getAllocatedBookings = async () => {
2          const [rows] = await dbPool.query(
3             "SELECT b.request_id, b.client_id, b.start_date, b.start_time,
                  b.end_time, h.hall_id, h.hall_name
4             FROM allocated_bookings a, booking_request b, lecture_hall h
5             WHERE a.hall_id = h.hall_id AND a.request_id = b.request_id"
6          );
7          return rows;
8       } // the rest of the code
```

Listing 6.1: Complex queries with table joins

line 1 - An asynchronous function getAllocatedBookings is declared.

line 2 - A destructured variable `rows` stores the result returned by querying the database using the query() method.

line 3 to 5 - A string representing the SQL query.

line 7 - The result obtained from the query that can be used by other application components

- **Secured integration with the server**: The **Node.js** server-side application communicates with the database through the **mysql2** library. Prepared statements can be created for any query to prevent SQL injections.

6.5 Version Control and Deployment

- **GitHub:** Stores the source code and runs testing and deployment worklflows using **GitHub Actions**.
- **Netlify:** Hosts the deployed **React.js** application.
- **Render:** Hosts the deployed **Express.js** application.
- **AWS:** Runs a VM that hosts the database.

7. TESTING

7.1 Test-Driven Development Approach

Unit and functional tests were written and deliberately run to fail to identify bugs and error during the development phase. Then, the implementation code was developed. Once the tests passed, the code was refactored for improved structure and readability.

7.1.1 Unit Tests

Tests were written using **Jest Framework** and were run primarily on the allocation algorithms. Mock bookings and halls was used to test the algorithms, ensuring that the tests could be run without needing a connection to the actual database or altering its data.

7.1.2 Functional Testing

Playwright was used for end-to-end testing, allowing automated browser interactions and verifying that the application behaves as expected from the user's perspective.

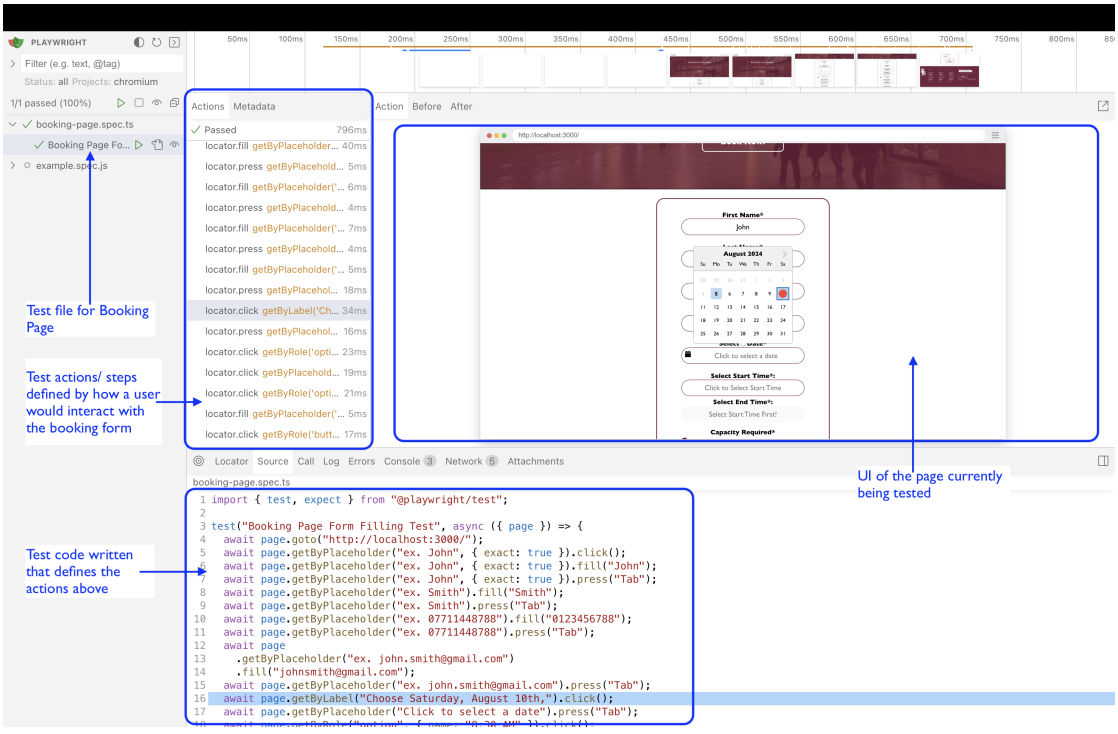


Figure 7.1 ***Functional Testing:*** A test running on the Playwright interface.

Feature	Description	Test Case	Expected
Submit a booking request	Allow user to select their requirements and submit a new booking request	Create a new booking request with valid inputs	Booking request created successfully
Allocation algorithm	Test if algorithms produce expected results and show on GUI	Create various booking requests and execute different algorithms	Each algorithm shows allocation on GUI with expected results
Staff user authentication	The system should log in a resource manager with valid credentials	Access the system with valid credentials of a resource manager	The user is redirected to the resource management page
Check Booking	Existing client can use their client ID and email to check their booking	Access the system with valid credentials of a client	The user is redirected to the check booking page and can view bookings in card or calendar mode

Table 7.1: Functional Test Cases

7.2 Automated Testing Pipeline

To ensure both the front- and back-end applications run as intended after code changes, a testing-and-deployment pipeline was set up using **GitHub Actions**. Whenever there is a new commit pushed to the repository, a new version of the application will only be deployed online after the unit and functional tests passed. See section 9.6. for the code setup.

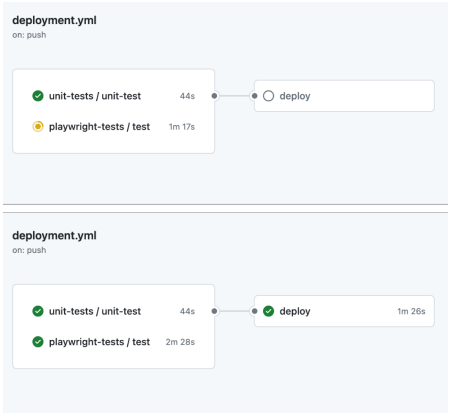


Figure 7.2 ***CI/CD pipeline:*** Deployment only starts when both tests passed.

8. CONCLUSION

8.1 Personal Reflection

The project successfully implemented and tested multiple algorithms for automating hall allocation. Although the backtracking algorithm yields the highest possible profit, it quickly becomes impractical as the number of bookings or halls increases. Attempts were made to adapt this into a dynamic programming algorithm; however, the complexity of managing time overlaps and memoising specific booking combinations meant that these efforts did not succeed.

An ILP approach proved to be both efficient and reliable for the majority of use cases. It generated allocation results comparable to those produced by the backtracking algorithm, but with a significantly reduced processing time. The crucial breakthrough in effectively utilising this package was in setting booking timeslots as constraints for each hall.

BOOKBBK evolved from a proof-of-concept during the proposal stage to a prototype with basic structures, and finally to the fully deployed online application. It brought together the knowledge and skills acquired throughout the MSc Computer Science course, while also pushing beyond into areas like cloud services and online deployment, where experience was previously limited.

8.2 Future Steps

Although the allocation problem is NP-complete, it is possible to explore more various approaches for finding near-optimal solutions, also known as *approximation algorithm* (Cormen et al. 2022). ILP was one of the the algorithms used.

Additionally, while the current system only permits booking submissions before the start of term, a future enhancement could introduce a standalone booking feature. This would allow clients to book any remaining available time slots after the primary term-time allocations have been completed. Another planned improvement is the development of an admin dashboard to oversee and export hall allocation data and statistics across different semesters.

REFERENCES

Alghamdi, Husam et al. (Dec. 2020). "A Review of Optimization Algorithms for University Timetable Scheduling". In: DOI: 10.48084/etasr.3832.

Burke, E. and D. Varley (Apr. 2000). "Automating Space Allocation in Higher Education E.K. Burke, D.B. Varley". In.

Cormen, Thomas H. et al. (2022). *Introduction to algorithms*. The MIT Press.

Garey, Michael R. (1979). *Computers and Intractability: A Guide to the Theory of NP-completeness*. W.H. Freeman and Co Ltd. ISBN: 0716710455.

GeeksforGeeks (Feb. 2023). *Job sequencing problem*. URL: https://www.geeksforgeeks.org/job-sequencing-problem/.

Kumar, Shive (2010). *The Purpose and function of a Data Access Layer*. URL: http://www.matlus.com/the-purpose-and-function-of-a-data-access-layer/ (visited on 08/19/2024).

Miller, Holmes E., William P. Pierskalla, and Gustave J. Rath (1976). "Nurse Scheduling Using Mathematical Programming". In: *Operations Research* 24.5, pp. 857–870. DOI: 10.1287/opre.24.5.857. eprint: https://doi.org/10.1287/opre.24.5.857. URL: https://doi.org/10.1287/opre.24.5.857.

Müller, Tomáš and Roman Barták (Sept. 2001). "Interactive Timetabling". In: *CoRR* cs.PL/0109022. DOI: 10.48550/arXiv.cs/0109022.

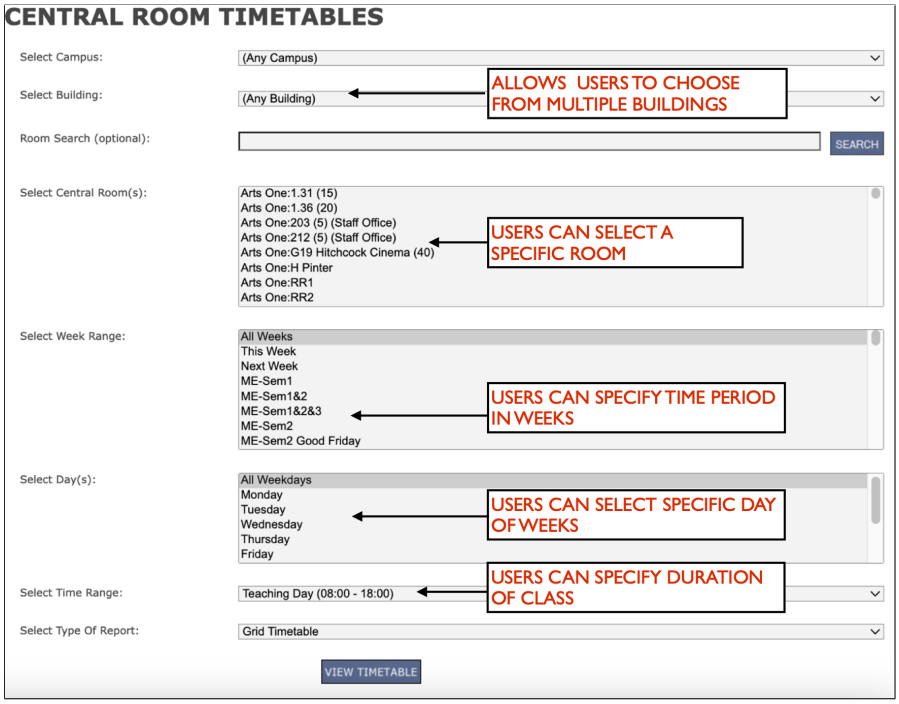
Rudová, Hana, Tomáš Müller, and Kenneth Murray (2011). "Complex university course timetabling". In: pp. 187–297. DOI: 10.1007/s10951-010-0171-3.

9. APPENDIX A

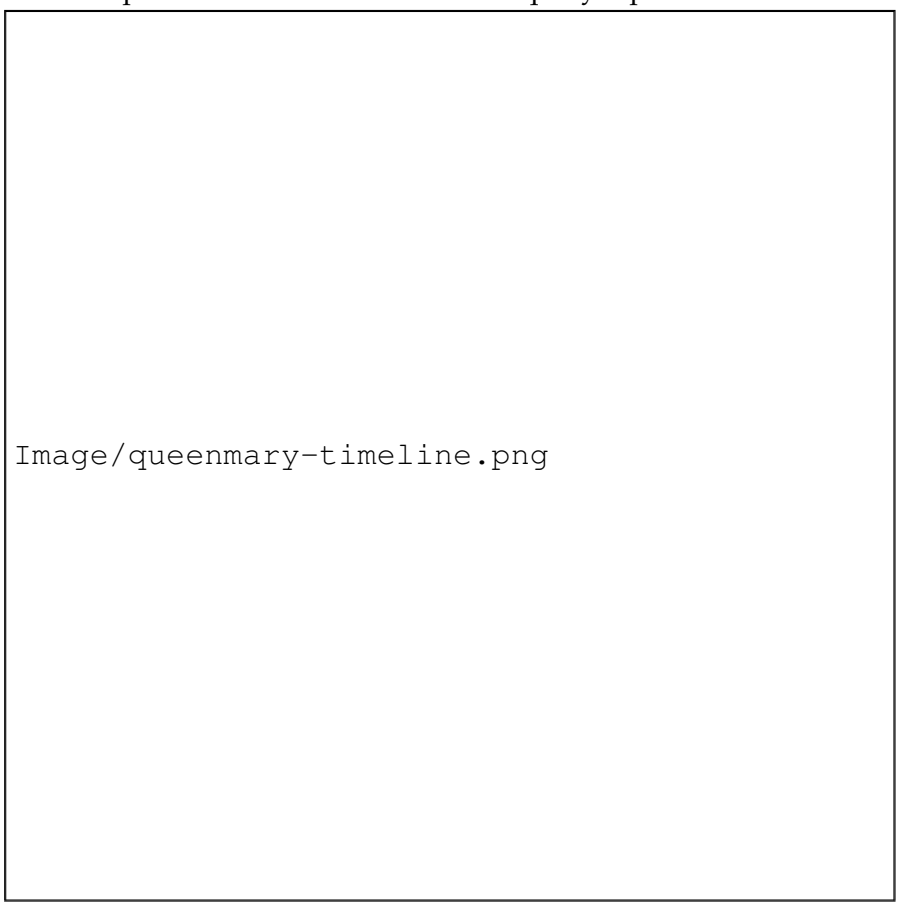
9.1 Precedent Software

This section shows precedent studies made on existing software which have inspired the design of **BOOKBBK**.

9.1.1 Timetable - Queen Mary University of London



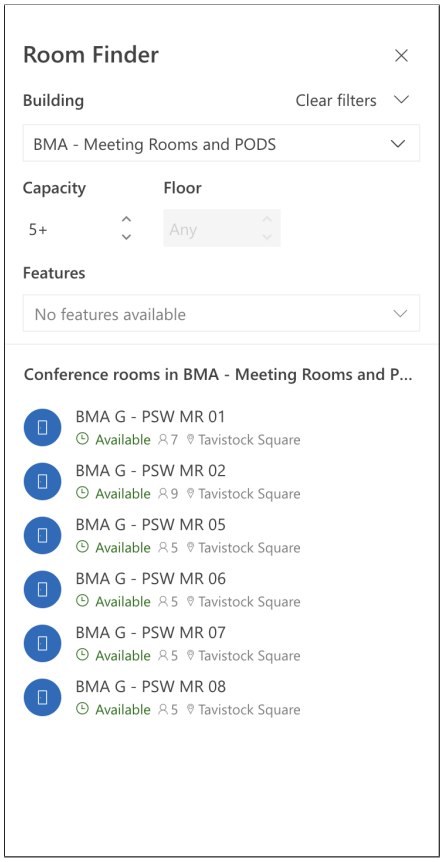
(a) A simple user interface with various query input fields.



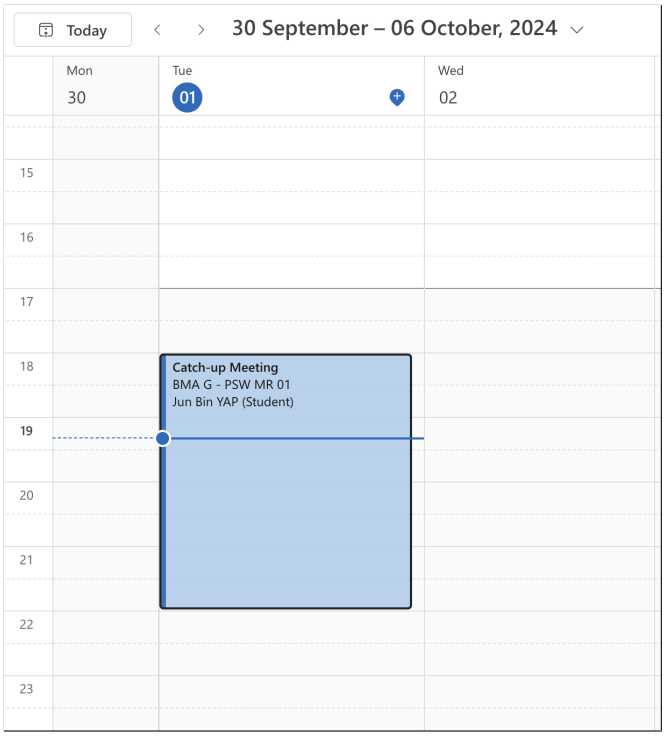
(b) A timetable view as a query result from.

Figure 9.1 An example of a timetable viewing application of a university

9.1.2 MS Teams Room Finder - Birkbeck, University of London



(a) A simple user interface that allows user to select a venue based on booking preferences.



(b) Successful booking reflected in personal calendar.

Figure 9.2 An example of a booking application of a university.

9.2 Software Development Methodology and Process

An iterative approach was employed where the development was broken down into phases/sprints. During each phase, prototypes of components were developed before refactoring and combining them into the final application.

9.2.1 Sprint 0 - Development Environment

Visual Studio Code is used as the main integrated development environment (IDE) for writing, compiling, testing, and debugging the software. All required software, tools, and package dependencies were installed for the purpose of developing each of the software tiers.

9.2.2 Sprint 1 - GUI Development

During this sprint, the focus was on developing the graphical user interface (GUI) for the application. For instance, a home page for booking submission and a resource management page for resource manager were created.

- **Component Design** - React library was used to create reusable UI components.
- **State Management** - State management within the application was handled using React's built-in hooks, avoiding the need for external libraries like Redux. This approach simplified the state management process.
- **Styling** - Styling was implemented using CSS modules to ensure modularity and maintainability. Additionally, libraries like `@mui/material` were used to enhance the visual appeal of the components.
- **Initial Error Handling** - Before submitting a booking form, user inputs are thoroughly validated on the front-end. This includes checking for required fields, valid data formats, and logical consistency (e.g., ensuring the end time is after the start time). If any validation errors are detected, appropriate error messages are displayed to the user immediately, preventing the submission of incorrect or incomplete data.

9.2.3 Sprint 2 - Server-side Development

The server-side development focused on setting up the backend to handle data processing and application logic.

- **API Design** - RESTful APIs were designed to allow communication between the front- and back-end. Endpoints were created for various

operations such as fetching booking data from the database, running greedy algorithms to allocate lecture halls, and retrieving allocation results.

- **Database Communication** A MySQL client, `mysql2` is installed as a dependency to communicate with the database. SQL queries can then be written in strings in Node.js environment and executed.
- **Error Handling** This included validating the payload from client booking submissions and returning appropriate error messages to the front-end application.

9.2.4 Sprint 3 - Relational Database

1. An entity-relationship diagram was created to help modelling the database schema.

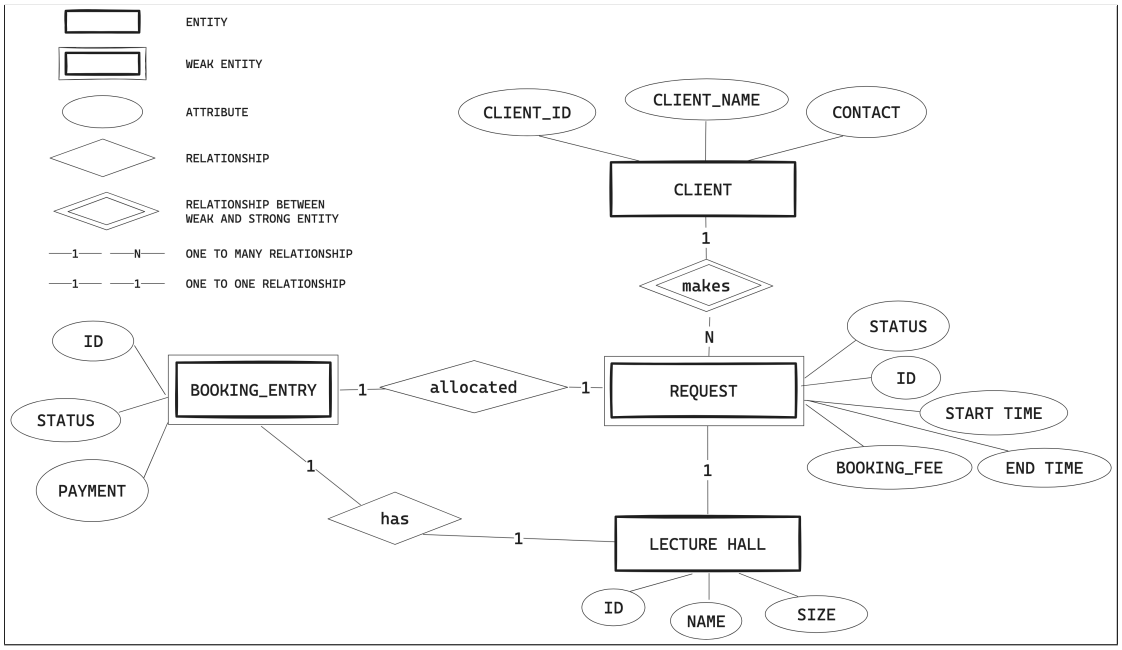


Figure 9.3 *Initial ER diagram*

A weak entity like the REQUEST table means that it cannot be identified without knowing the client to which the request belongs.

2. The schema created includes tables for clients, university staff, booking requests, lecture halls, and allocation results. Relationships between tables were defined using foreign keys to ensure data integrity.

9.2.5 Sprint 4 - Implementing Allocation Algorithms

This sprint focused on developing algorithms to allocate resources automatically. Two greedy algorithms, a random allocation algorithm and a

backtracking algorithm was developed and tested. Additionally, an algorithm using ILP was created using an external package, `jsLPSolver`.

9.2.6 Sprint 5 - Deploying BOOKBBK Online

Applications from different software tiers were deployed separately to prevent single point systematic failure. As this project requires a relatively low amount of storage and computations, online/cloud platforms that provide free-tier services were chosen to deploy the applications developed in the previous sprints.

9.2.7 Database Deployment

The database was created and run locally during the development phase. To deploy, all the data from the local database was backed up using `sqldump` command. Then, a **Ubuntu** virtual machine(VM) is set up in a **Elastic Compute Cloud(EC2)** from **Amazon Web Services(AWS)**, in order to install a **MySQL** client on the VM. Once the new client had imported the database backup, the back-end application could now connect to the online database using a public IPv4 address provided by the **EC2** instance.

9.2.8 Front-end Application Deployment

The React application was hosted on **Netlify** as a static application which can make network calls to the back-end application.

9.2.9 Back-end Application Deployment

As **Netlify** only provides hosting for static application, **Render** was chosen to host the server-side application as it supports continuous integration and deployment. When the repository of the project is updated, the application will be re-deployed.

9.3 Running Algorithms Using GUI

9.3.1 Backtracking Algorithm

As this approach employed recursive method to find all possible combinations and choose the most profitable allocations, the profit generated using this approach can be predicted to be the highest.

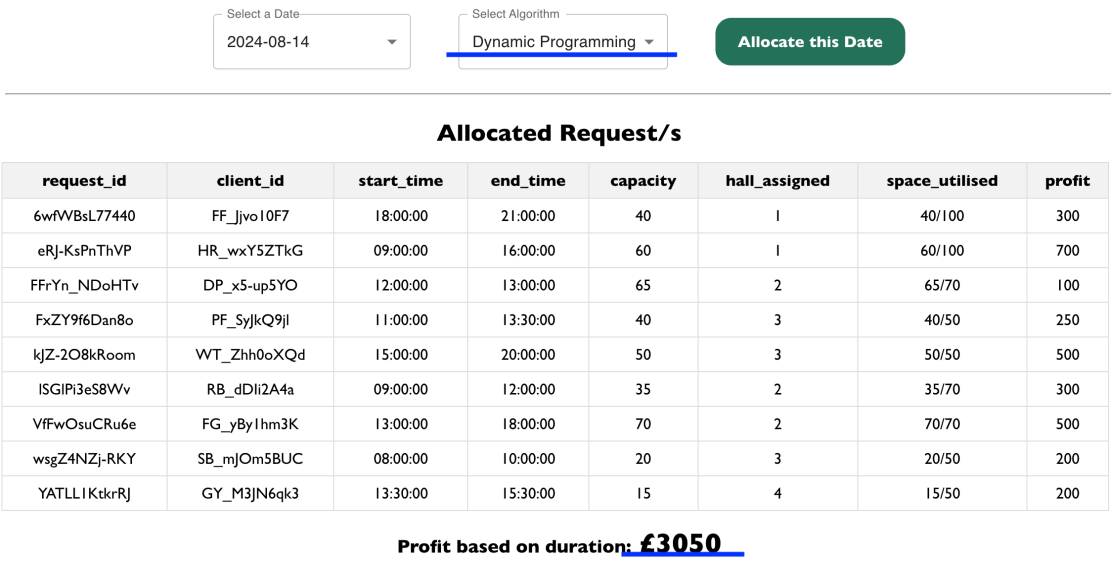


Figure 9.4 Allocation results using dynamic programming

9.3.2 Greedy - Earliest Start Time

Bookings with earlier start time are prioritised by the algorithm and it returned a profit almost as high as the dynamic programming approach. However, the high profit result is only by chance and likely not to be the same if run on different data set as bookings will have different booking duration and the individual profit is calculated based on that.

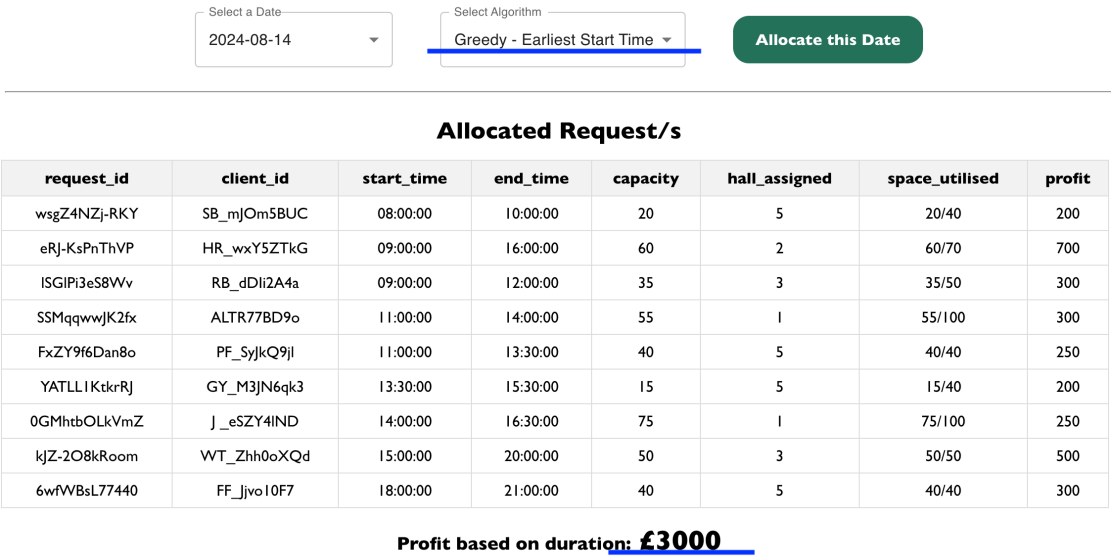


Figure 9.5 Allocation results based on earliest start time

9.3.3 Greedy - Longest Duration

Although the profit of each booking is directly proportional to the duration, it is observed here in the result that it has a lower profit than the previous two algorithms. This is due to the fact that even the longer bookings are preferred, there might be a situation where a single longer booking might be less profitable than 2 shorter bookings combined.

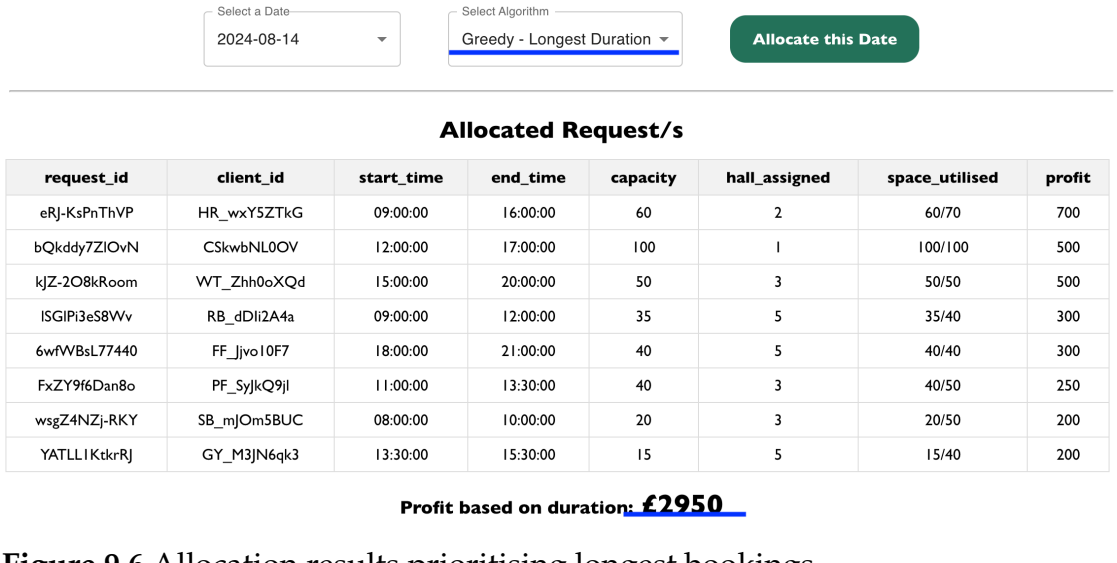
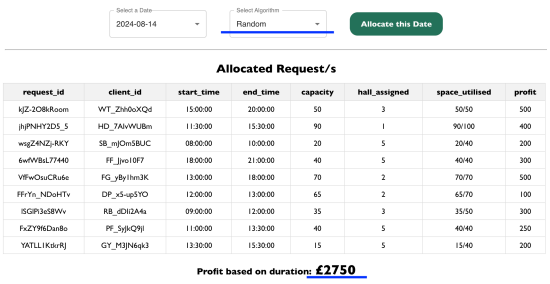


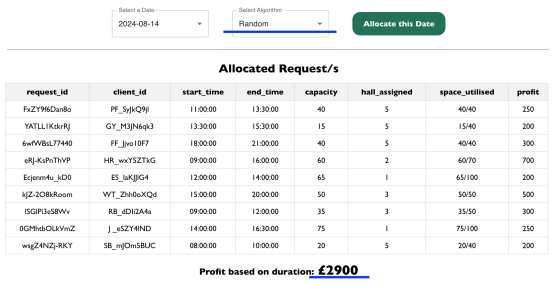
Figure 9.6 Allocation results prioritising longest bookings

9.3.4 Random Allocation

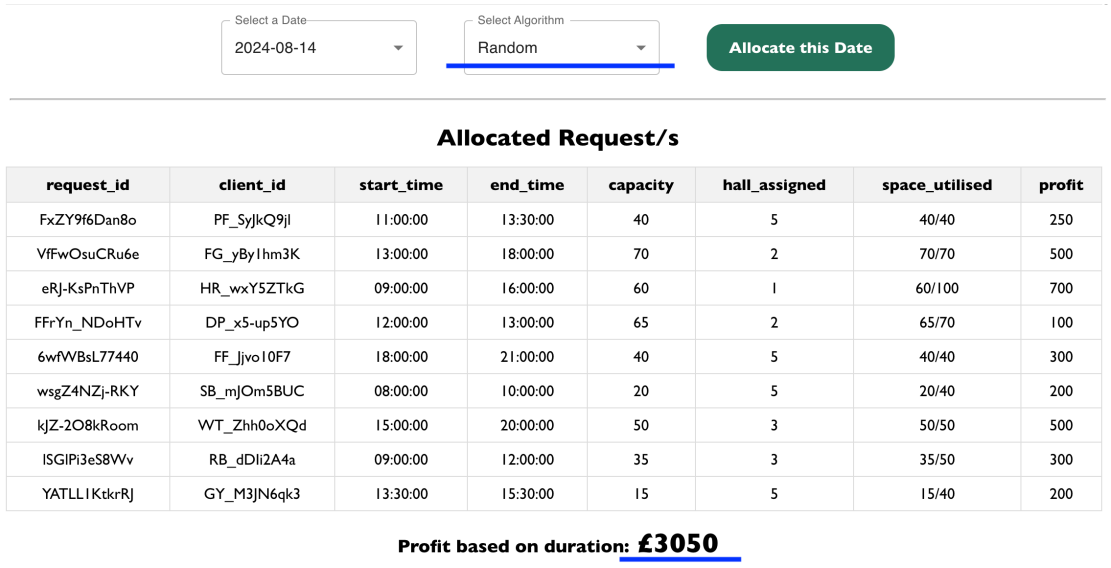
A random algorithm is the least reliable method as it generates different results every run. Though it might return the highest profit (See Figure 9.5), the same result as the dynamic programming approach, there is no way to know that if this is definitely the highest without also running other algorithms first to compare.



(a) Profit = £2750



(b) Profit = £2900



(c) Profit = £3050

Figure 9.7 Allocation results using random allocation

9.4 Sample Bookings For 5.4

booking_num	start_time	end_time	capacity
1	07:00:00	09:30:00	100
2	08:30:00	10:00:00	70
3	09:00:00	11:00:00	50
4	09:30:00	12:30:00	50
5	10:00:00	13:00:00	40
6	11:00:00	14:00:00	100
7	11:30:00	13:30:00	70
8	12:00:00	15:00:00	50
9	12:30:00	14:30:00	40
10	13:00:00	16:00:00	100
11	14:00:00	16:30:00	70
12	14:30:00	17:00:00	50
13	15:30:00	18:30:00	40
14	16:00:00	18:00:00	70
15	17:00:00	19:00:00	100

(a) *15 distinct bookings for Test 1*

booking_num	start_time	end_time	capacity
1	07:00:00	09:30:00	57
2	09:30:00	13:30:00	86
3	08:00:00	10:00:00	45
4	11:00:00	13:00:00	40
5	17:00:00	20:00:00	82
6	13:00:00	15:30:00	49
7	10:00:00	12:30:00	53
8	13:30:00	16:00:00	92
9	16:00:00	18:30:00	75
10	16:30:00	19:00:00	57
11	18:00:00	21:00:00	55
12	14:00:00	17:00:00	47
13	12:00:00	15:00:00	66
14	19:00:00	21:30:00	68
15	20:00:00	22:00:00	60

(b) *15 distinct bookings for Test 2*

booking_num	start_time	end_time	capacity
1	11:30:00	14:00:00	45
2	11:00:00	13:30:00	50
3	14:00:00	16:30:00	52
4	12:30:00	15:30:00	70
5	14:30:00	17:00:00	88
6	15:00:00	19:00:00	45
7	16:00:00	18:00:00	78
8	17:00:00	20:30:00	40
9	17:30:00	19:00:00	60
10	18:00:00	21:00:00	50
11	19:30:00	22:30:00	72
12	20:00:00	22:30:00	30
13	08:00:00	10:30:00	60
14	10:30:00	15:30:00	85
15	09:00:00	11:00:00	48

(c) *15 distinct bookings for Test 3*

Figure 9.8 *3 sets of 15 distinct bookings for testing ILP and Backtracking algorithms*

9.5 ILP Setup

9.5.1 Hall Time Constraints

Below is a constant variable that creates the available hours for each hall as resources. By making each time slot have a maximum resource of 1 unit, it prevents multiple bookings from taking one same time slot which would result in double-booking.

```
const LPSOLVER_HALL_CONSTRAINTS = {
   "h6": { max: 1 },
   "h6.5": { max: 1 },
   "h7": { max: 1 },
   "h7.5": { max: 1 },
   "h8": { max: 1 },
   "h8.5": { max: 1 },
   "h9": { max: 1 },
   "h9.5": { max: 1 },
   "h10": { max: 1 },
   "h10.5": { max: 1 },
   "h11": { max: 1 },
   "h11.5": { max: 1 },
   "h12": { max: 1 },
   "h12.5": { max: 1 },
   "h13": { max: 1 },
   "h13.5": { max: 1 },
   "h14": { max: 1 },
   "h14.5": { max: 1 },
   "h15": { max: 1 },
   "h15.5": { max: 1 },
   "h16": { max: 1 },
   "h16.5": { max: 1 },
   "h17": { max: 1 },
   "h17.5": { max: 1 },
   "h18": { max: 1 },
   "h18.5": { max: 1 },
   "h19": { max: 1 },
   "h19.5": { max: 1 },
   "h20": { max: 1 },
   "h20.5": { max: 1 },
   "h21": { max: 1 },
   "h21.5": { max: 1 },
};
```

Listing 9.1: how the available hours are modelled as constraints

9.5.2 Example Booking as Variable

Each booking has its ID as key and requirements as values. The start and end time were divided into units of time that match the format of the time constraints.

```
1  {
2     '336FENFpXd7Q': { 'h14': 1, 'h14.5': 1, 'h15': 1, 'h15.5': 1, capacity: 90,
          profit: 200 },
3     '4qH6mtzqZed4': { 'h18': 1, 'h18.5': 1, capacity: 90, profit: 100 },
4     '57-e-2dnGU0F': { 'h16': 1, 'h16.5': 1, 'h17': 1, 'h17.5': 1, capacity: 90,
          profit: 200 },
5  }
```

Listing 9.2: Event durations are broken down into one unit of time

9.5.3 Example Solver Model

This is a sample model that will be created for every single hall.

```
1  model = {
2      optimize: { profit: "max", capacity: "max" },
3      opType: "max"
4     "constraints": {
5        "capacity": 150,
6        "h6": { max: 1 },
7        "h6.5": { max: 1 },
8        "h7": { max: 1 },
9        "h7.5": { max: 1 },
10       "h8": { max: 1 },
11       "h8.5": { max: 1 },
12       "h9": { max: 1 },
13       // ...
14       // the rest of the time units
15    }
16    "variables": {
17       '336FENFpXd7Q': { 'h14': 1, 'h14.5': 1, 'h15': 1, 'h15.5': 1, capacity: 90,
             profit: 200 },
18       '4qH6mtzqZed4': { 'h18': 1, 'h18.5': 1, capacity: 90, profit: 100 },
19       '57-e-2dnGU0F': { 'h16': 1, 'h16.5': 1, 'h17': 1, 'h17.5': 1, capacity: 90,
             profit: 200 },
20    }
21 };
```

Listing 9.3: Event durations are broken down into one unit of time

line 2 - The objective is to maximise profit and space utilisation (halls should prefer bookings that make the most use of their capacity). Without specifying `capacity: {"max"}`, the model will disregard how much space is a booking occupying for a hall and only focus on profit.

line 4 to 13 - The resource available. To be used as a resource, the capacity is the actual capacity of the hall multiplied by the number of bookings. In this case for a hall of size 50, it is multipled by 3 as there are 3 bookings to be allocated.

line 15 to 18 - All bookings that have yet to be allocated.

9.6 CI/CD Pipeline

Below is a configuration file to setup the CI/CD pipeline on Github Actions. There are 3 jobs in the workflow: `unit-tests`, `playwright-tests` and `deploy`. The `deploy` job needs both unit and functional tests to pass first as a condition to run.

```yaml
name: Deploy To Netlify
on:
  push:
    paths-ignore:
      - "README.md"

jobs:
  unit-tests:
    uses: ./.github/workflows/unit-tests.yml
  playwright-tests:
     uses: ./.github/workflows/playwright.yml

deploy:
  needs: [unit-tests, playwright-tests]
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node
      uses: actions/setup-node@v4

    - name: Install Dependencies
      run: cd client && npm ci && npm run build

    - name: Install Netlify CLI
      run: npm install -g netlify-cli

    - name: Deploy to Netlify
      env:
        NETLIFY_AUTH_TOKEN: ${{ secrets.NETLIFY_AUTH_TOKEN }}
        NETLIFY_SITE_ID: ${{ secrets.NETLIFY_SITE_ID }}
      run: netlify deploy --prod --dir=client/build --site=$NETLIFY_SITE_ID
```

Listing 9.4: YAML file to configure the automation on Github Actions

10. APPENDIX B

BOOKBBK GitHub Repository

`https://github.com/junyap95/university-resource-allocation`

BOOKBBK Web Application

`https://bookbbk.netlify.app/`